



Facultad de Ingeniería de Sistemas e Informatica

“Gestor de Biblioteca”

Autor (es):

- Ramos Valdivia Gerardo Adrian, U23273502
- Rojas Gamonal Samir Angel, U23243868
- Salazar Camacho Sandro Piero, U23266038
- Tapia Canta Adriana Lucía, U23202658

Curso:

Herramientas de Desarrollo

Docente:

Jose Luis Milla Flores

Sección:

23280

Lima, Perú 2026

Agradecimiento

Agradecemos al docente del curso por su orientación durante el desarrollo de este proyecto y los conocimientos adquiridos, así como a nuestra universidad por brindarnos los recursos necesarios para su realización.

Dedicatoria

Dedicamos este trabajo a nuestras familias y a todas las personas que apoyaron y continúan respaldando nuestro proceso de formación académica.

Abstract

The present project involves the design and implementation of a web platform for personalized digital library management, developed using the Spring Boot framework and a modular architecture. The system enables authenticated users to interact with a public book API to search, leave comments, catalog literary works into customized lists (e.g., "to read", "completed"). The core of the application lies in the persistence of bibliographic metadata and the ability for users to record ratings and critical reviews, facilitating a detailed tracking of their reading activity. The database design optimizes the relationship between users and books through an associative table that ensures comment integrity. The implementation stands out for its use of WebClient for asynchronous consumption of external services and a client-side image management strategy to optimize server performance.

Keywords: Spring Boot, Book API, JPA, Data Persistence, Bibliographic Management, WebClient, Software Architecture.

Resumen

El presente proyecto consiste en el diseño e implementación de una plataforma web orientada a la gestión personalizada de bibliotecas digitales, desarrollada bajo el framework Spring Boot y una arquitectura modular. El sistema permite a los usuarios autenticados interactuar con una API pública de libros para la búsqueda, interacción de comentarios y catalogación de obras en listas personalizadas (e.g., "por leer", "completado"). El núcleo de la aplicación reside en la persistencia de metadatos bibliográficos y la capacidad de los usuarios para registrar calificaciones y reseñas críticas, facilitando un seguimiento detallado de su actividad lectora. El diseño de la base de datos optimiza la relación entre usuarios y libros

mediante una tabla de asociación que garantiza la integridad de los comentarios. La implementación destaca por el uso de WebClient para el consumo asíncrono de servicios externos y una estrategia de gestión de imágenes basada en el lado del cliente para optimizar el rendimiento del servidor.

Palabras clave: Spring Boot, API de Libros, JPA, Persistencia de Datos, Gestión Bibliográfica, WebClient, Arquitectura de Software.

Contenido

1.	Realidad problemática	8
2.	Justificación.....	8
2.1.	Relevancia Social	8
2.2.	Relevancia Económica y Científica	9
3.	Objetivos	9
3.1.	Objetivo general	9
3.2.	Objetivos específicos.....	9
4.	Marco teórico	10
4.1.	Estado del arte.....	10
4.2.	Memoria y proceso de aprendizaje	11
4.3.	Lectura digital y comprensión	11
4.4.	Gestión del conocimiento personal	12
4.5.	Lectura digital y pensamiento crítico	13
5.	Metodología.....	13
6.	Cronograma	14
7.	Desarrollo de proyecto.....	14
7.1.	Aplicación.....	14
7.1.1.	Arquitectura de la aplicación	15
7.1.2.	Módulos implementados	15
7.1.3.	Controladores REST.....	16
7.1.4.	Documentación API con Swagger	16
7.2.	Base de datos.....	17
7.2.2.	Tablas principales	17
7.2.3.	Relaciones entre tablas.....	18
7.3.	Uso de herramienta Git y Github	20
7.3.1.	Visual Studio Code y otras herramientas	20
7.3.2.	Ciclo de Trabajo y Estados	22
7.3.3.	Gestión de ramas y colaboración.....	23

7.3.4.	Gestión Ágil con GitHub Projects e Issues	27
7.3.5.	Resolución de conflictos	30
7.3.6.	Control de versiones y tags	34
7.3.7.	Revisiones, sugerencias y comentarios	36
7.3.8.	Integración Continua con GitHub Actions	39
7.3.9.	Calidad de código con SonarCloud	41
7.3.10.	Pipeline CI/CD.....	42
7.3.11.	Dockerización de la aplicación	43
7.3.12.	Docker Multi-Stage	45
7.3.13.	Seguridad de dependencias con Snyk	46
7.3.14.	Escaneo de vulnerabilidades de imágenes con Trivy	48
7.3.15.	Despliegue de la aplicación en Azure	48
7.3.15.1.	<i>Arquitectura de despliegue</i>	48
7.3.15.2.	<i>Despliegue del Frontend</i>	49
7.3.15.3.	<i>Despliegue del Backend</i>	51
7.3.15.4.	<i>Despliegue de la Base de Datos</i>	53
7.3.15.5.	<i>Validación del despliegue</i>	54
7.4.	Herramientas de organización y desarrollo de proyectos	56
7.4.1.	Gestión de incidentes con Jira Service Management	61
7.4.2.	Automatización del flujo de incidencias.....	63
7.4.3.	Integración Jira-GitHub.....	65
7.5.	Resultados obtenidos.....	67
7.5.1.	Funcionamiento del sistema	67
7.5.2.	Evaluación del pipeline DevOps	71
8.	Conclusiones	71
9.	Referencias bibliográficas	72

1. Realidad problemática

En la era de la sobreinformación digital, la retención de conocimientos provenientes de la lectura profunda ha disminuido drásticamente. Según artículos del CERLALC (Centro Regional para el Fomento del Libro en América Latina y el Caribe), aunque el uso de textos digitales en la población peruana ha incrementado de 20,0 % al 25,6 % en los últimos años, la capacidad de síntesis y memoria crítica de los lectores jóvenes puede haber disminuido. De la teoría del 'Ahorro de Reaprendizaje' de Ebbinghaus, se puede deducir que, si un usuario invirtió 10 horas en comprender un libro, pero meses después necesita las mismas 10 horas para recordarlo, su ahorro es del 0%, lo que quiere decir que olvidó el libro por completo.

El problema detectado no es la falta de lectura, sino la falta de gestión del conocimiento personal. Los lectores actuales carecen de una herramienta centralizada que combine la organización bibliográfica con la capacidad de almacenar resúmenes y opiniones propias de manera estructurada que les permita recordar lo aprendido. Las aplicaciones genéricas de notas son demasiado dispersas, y las redes sociales de libros se enfocan más en la calificación que en la reflexión profunda del lector.

2. Justificación

2.1. Relevancia Social

Este proyecto fomenta la alfabetización funcional. Al facilitar un espacio para opiniones y resúmenes, se incentiva al usuario a procesar la información en lugar de consumirla pasivamente. Socialmente, esto contribuye a ciudadanos con mayor capacidad crítica frente a problemáticas.

2.2. Relevancia Económica y Científica

Desde una perspectiva científica, el proyecto se alinea con las teorías de la psicología cognitiva sobre la *repetición espaciada* y el *auto-procesamiento* de la información.

Económicamente, el desarrollo de este software permite optimizar el tiempo de investigadores y estudiantes, quienes suelen gastar horas valiosas buscando apuntes dispersos y tratando de recordar lo leído. Centralizar esta información reduce la *deuda cognitiva* y mejora la productividad académica.

La investigación y desarrollo de esta solución es de suma urgencia. Con el aumento de contenidos cortos en redes sociales, la lectura de libros se ha convertido en un acto de resistencia intelectual. Si no proporcionamos herramientas que ayuden a fijar esos conocimientos de forma digital y organizada, corremos el riesgo de que la lectura profunda se convierta en una actividad de consumo superficial y efímero, perdiendo el valor educativo que los libros han aportado históricamente a la civilización.

3. Objetivos

3.1. Objetivo general

Diseñar e implementar un software de gestión de libros favoritos que integre una API externa para organizar lecturas, almacenar resúmenes y opiniones, con el fin de fortalecer la memoria crítica y la gestión del conocimiento en la era digital.

3.2. Objetivos específicos

- Analizar las necesidades de los lectores en relación con la gestión del conocimiento personal, elaborando un informe de requerimientos funcionales y no funcionales.

- Diseñar la arquitectura del sistema y la base de datos relacional que soporte usuarios, libros, listas y reseñas, documentando un esquema técnico documentado.
- Implementar la integración con una API externa de libros para permitir la búsqueda y registro de títulos, desarrollando un módulo funcional de conexión.
- Desarrollar un prototipo interactivo (MVP) que permita a los usuarios crear listas de favoritos, añadir reseñas y almacenar resúmenes, entregando una versión inicial utilizable dentro del curso.

4. Marco teórico

El presente marco teórico tiene como finalidad sustentar el desarrollo del sistema propuesto a partir de teorías e investigaciones relacionadas con la lectura digital, la memoria y la gestión del conocimiento. A través del análisis del estado del arte y conceptos clave, se busca comprender cómo han cambiado los hábitos de lectura y qué herramientas pueden contribuir a mejorar la retención y organización de la información.

4.1. Estado del arte

En los últimos años, la lectura se ha ido digitalizando progresivamente, permitiendo a los usuarios acceder a contenidos desde distintos dispositivos. Este cambio ha facilitado el acceso a la información, pero también ha generado nuevos retos en la comprensión y retención de lo leído. Diversos estudios recientes indican que la lectura en entornos digitales tiende a ser más fragmentada, ya que los usuarios interactúan con múltiples fuentes al mismo tiempo. Esto puede afectar la concentración y dificultar la comprensión profunda del contenido. Asimismo, Cordón y Jarvio (2015) señalan que la transformación digital de la lectura ha modificado los hábitos de los lectores, quienes ahora priorizan la rapidez y la accesibilidad sobre el análisis detallado. Por otro

lado, investigaciones más recientes evidencian que la lectura digital puede mejorar ciertos aspectos del aprendizaje si se utilizan herramientas adecuadas, como sistemas de organización, anotación y seguimiento del contenido (Pérez et al., 2023). También se ha observado que los jóvenes consumen mayor cantidad de información en formato digital, pero muchas veces sin procesarla de manera crítica (Figuerola et al., 2016). Esto refuerza la idea de que el problema no es el acceso a la lectura, sino la forma en que se gestiona la información. En conjunto, estos estudios evidencian la necesidad de desarrollar herramientas que permitan no solo acceder a libros, sino también organizar, reflexionar y retener el conocimiento obtenido.

4.2. Memoria y proceso de aprendizaje

La memoria es un proceso fundamental en el aprendizaje, ya que permite almacenar y recuperar información. Sin embargo, la retención del conocimiento depende de cómo se procese la información. Baddeley (2015) explica que la memoria humana requiere procesos activos para consolidar la información, como la repetición, la organización y la asociación de ideas. Esto implica que la lectura pasiva no es suficiente para garantizar el aprendizaje. En el contexto actual, donde existe una gran cantidad de información disponible, las personas tienden a olvidar con mayor facilidad aquello que no utilizan o refuerzan. Por ello, es necesario contar con herramientas que faciliten la organización y recuperación del conocimiento.

4.3. Lectura digital y comprensión

La lectura digital ha introducido nuevas formas de interacción con el texto, modificando los procesos tradicionales de comprensión. A diferencia de la lectura en formato impreso, los entornos digitales suelen presentar información de manera no lineal, lo que obliga al lector a tomar decisiones constantes sobre qué leer, cómo hacerlo y en qué orden. Según Delgado, Vargas, Ackerman y Salmerón (2018), la comprensión lectora puede verse afectada en medios

digitales debido a factores como la distracción, la navegación hipertextual y la menor profundidad en el procesamiento de la información. Estos elementos pueden generar una lectura más rápida, pero menos reflexiva. Además, destacan que la comprensión en entornos digitales mejora cuando el lector aplica estrategias metacognitivas, como planificar la lectura, monitorear la comprensión y evaluar la información encontrada. Asimismo, la Organización para la Cooperación y el Desarrollo Económicos (OCDE, 2019) señala que los estudiantes con mejores habilidades de navegación digital logran interpretar y relacionar información de múltiples fuentes con mayor eficacia. En este contexto, la comprensión lectora en entornos digitales no depende únicamente del acceso a la información, sino de la capacidad del usuario para gestionarla activamente. Por ello, resulta fundamental contar con herramientas que faciliten la organización, el análisis y la reflexión del contenido leído.

4.4. Gestión del conocimiento personal

En la era digital, las personas manejan grandes cantidades de información, lo que hace necesario contar con estrategias para organizarla. La gestión del conocimiento personal se refiere al proceso mediante el cual un individuo recopila, organiza y utiliza información de manera eficiente. En este contexto, las herramientas digitales juegan un papel fundamental. Según Montoya et al. (2019), los sistemas digitales pueden facilitar la organización del conocimiento cuando permiten estructurar la información y acceder a ella fácilmente. En el caso de la lectura, esto implica contar con una herramienta que permita registrar libros, almacenar resúmenes, guardar opiniones y organizar contenido de manera estructurada. El sistema propuesto responde a esta necesidad, ofreciendo una solución centralizada para la aplicación del conocimiento lector.

4.5. Lectura digital y pensamiento crítico

La lectura no solo implica comprender información, sino también analizarla y reflexionar sobre ella. Investigaciones recientes indican que la lectura digital puede contribuir al desarrollo del pensamiento crítico cuando se acompaña de estrategias de análisis y reflexión (González-Ramírez et al., 2020). Sin embargo, el consumo rápido de información en entornos digitales puede limitar este proceso, favoreciendo una lectura superficial. Por ello, es importante fomentar prácticas como la escritura de resúmenes y opiniones, las cuales permiten procesar la información de manera más profunda.

5. Metodología

El presente proyecto corresponde a una investigación de tipo aplicada, ya que tiene como finalidad desarrollar una solución tecnológica orientada a mejorar la gestión del conocimiento en la lectura digital. Asimismo, se adopta un enfoque cualitativo, debido a que se busca comprender el problema a partir del análisis del comportamiento del usuario frente al uso de herramientas digitales, especialmente en la organización y retención de la información leída.

Para la recolección de información se empleará la técnica de observación, la cual permitirá analizar cómo los usuarios interactúan con aplicaciones actuales de lectura y toma de notas, identificando dificultades en la organización, recuperación y comprensión de la información. Esta técnica se complementará con la revisión documental de estudios académicos relacionados con la memoria, la lectura digital y la gestión del conocimiento, lo que permitirá sustentar teóricamente el desarrollo del proyecto.

En cuanto a los materiales e instrumentos, se utilizará una computadora personal, el entorno de desarrollo Visual Studio Code, el framework Angular para el desarrollo del frontend,

Spring Boot para la construcción del backend y PostgreSQL como sistema de gestión de base de datos. Asimismo, se integrará una API externa de libros que permitirá la búsqueda y registro de información bibliográfica dentro del sistema.

El diseño del sistema se desarrollará bajo un enfoque de ingeniería de software iterativo, basado en la construcción de un producto mínimo viable (MVP). Este proceso incluirá la etapa de análisis, en la que se identificarán los requerimientos del sistema; diseño, donde se definirá la arquitectura cliente-servidor y el modelo de base de datos; implementación, en la que se desarrollarán los componentes del frontend y backend, además de la integración con la API externa; y finalmente la etapa de pruebas, donde se verificará el correcto funcionamiento de las funcionalidades principales del sistema.

Finalmente, la validación del sistema se realizará comprobando que el usuario pueda buscar libros mediante la API, registrar sus libros favoritos, almacenar resúmenes y opiniones, así como verificar que la información se guarde correctamente en la base de datos. También se evaluará que el sistema sea funcional y fácil de utilizar, asegurando que cumpla con el objetivo de mejorar la gestión del conocimiento del usuario.

6. Cronograma

El cronograma está colocado el apartado de anexos.

7. Desarrollo de proyecto

7.1. Aplicación

7.1.1. Arquitectura de la aplicación

El desarrollo de la aplicación se basa en el framework Spring Boot, aprovechando su ecosistema para crear una herramienta escalable. Se ha implementado una arquitectura en capas, lo que permite una clara separación de responsabilidades y facilita el mantenimiento del código. La aplicación está compuesta por capas compuesta por controladores, servicios, repositorios y entidades. Asimismo, se implementaron DTOs (Data Transfer Objects) para optimizar la transferencia de información entre el backend y el cliente.

7.1.2. Módulos implementados

Durante el avance 2 se desarrollaron distintos módulos funcionales de la aplicación, entre ellos están los siguientes.

Módulo de autenticación

Se implementó un sistema de registro e inicio de sesión de usuarios utilizando Spring Security. Las contraseñas son cifradas mediante el algoritmo BCrypt antes de almacenarse en la base de datos, garantizando la protección de la información sensible.

Módulo de gestión de libros

El sistema permite buscar libros en tiempo real mediante la integración con la API Open Library. La aplicación consume los datos externos utilizando WebClient y transforma las respuestas JSON en objetos DTO para su posterior procesamiento.

Módulo de listas personalizadas

Los usuarios pueden crear listas personalizadas como “Favoritos”, “Pendientes” o “Leídos”, permitiendo organizar sus lecturas de manera flexible.

Módulo de reseñas

Se implementó un sistema de reseñas donde cada usuario puede registrar comentarios y calificaciones asociadas a un libro específico.

7.1.3. Controladores REST

Se desarrollaron controladores REST para gestionar las operaciones principales del sistema mediante endpoints HTTP.

Los controladores fueron probados utilizando Postman y Swagger UI para verificar el correcto funcionamiento de las operaciones.

7.1.4. Documentación API con Swagger

Con el objetivo de mejorar la documentación técnica del sistema, se integró Swagger en el proyecto Spring Boot. Esta herramienta permite generar automáticamente una interfaz web donde se visualizan todos los endpoints disponibles, los parámetros requeridos y las respuestas del sistema. La documentación puede visualizarse desde:

<http://localhost:8080/swagger-ui/index.html>

- **Gestión de Versiones y Colaboración:** El ciclo de vida del desarrollo se gestionó mediante Git, utilizando GitHub como repositorio centralizado. Esto permitió mantener un historial detallado de cambios, la gestión de ramas para nuevas funcionalidades y la implementación de un flujo de trabajo profesional para la integración de código.
- **Consumo de Servicios Externos:** Para la obtención de datos bibliográficos, la aplicación se integra con la API de Open Library. Esta comunicación se realiza de

forma asíncrona mediante WebClient, configurado con un encabezado User-Agent personalizado para garantizar una mayor tasa de transferencia. La lógica de negocio está diseñada para procesar respuestas JSON complejas y transformarlas en objetos de transferencia de datos (DTOs) optimizados.

- **Gestión de Sesiones y Seguridad:** Se incluye un módulo de seguridad para el registro y autenticación de usuarios. Las contraseñas son procesadas mediante el algoritmo de hashing BCrypt antes de su almacenamiento, garantizando la confidencialidad de la información.
- **Lógica de Negocio de Catalogación:** El sistema permite al usuario buscar obras en tiempo real, pero solo persiste la información en la base de datos local cuando el usuario decide agregar un libro a una de sus listas personales. Esto evita el almacenamiento innecesario de datos y optimiza los recursos del servidor.

7.2. Base de datos

El sistema utiliza PostgreSQL como gestor de base de datos relacional. La persistencia se implementó mediante Spring Data JPA e Hibernate, permitiendo mapear las entidades del sistema a tablas relacionales.

7.2.1. Modelo relacional

El modelo relacional fue diseñado para mantener la integridad y trazabilidad de la información generada por los usuarios.

7.2.2. Tablas principales

Tabla Usuarios

Almacena la información de autenticación y perfil de cada usuario.

Tabla Libros

Contiene los libros registrados por los usuarios, almacenando información bibliográfica básica y el ISBN como identificador único.

Tabla Listas

Permite registrar listas personalizadas creadas por cada usuario.

Tabla Reseñas

Relaciona usuarios y libros, almacenando comentarios y calificaciones.

Tabla LibroEnLista

Permite registrar listas personalizadas creadas por cada usuario.

Tabla Actividad

Permite registrar acciones del usuario y tener un historial de ello.

7.2.3. Relaciones entre tablas

El sistema de persistencia utiliza PostgreSQL, gestionado a través de Spring Data JPA e Hibernate. El diseño se centra en un modelo relacional que permite una trazabilidad completa de la actividad del usuario sin redundancia de datos.

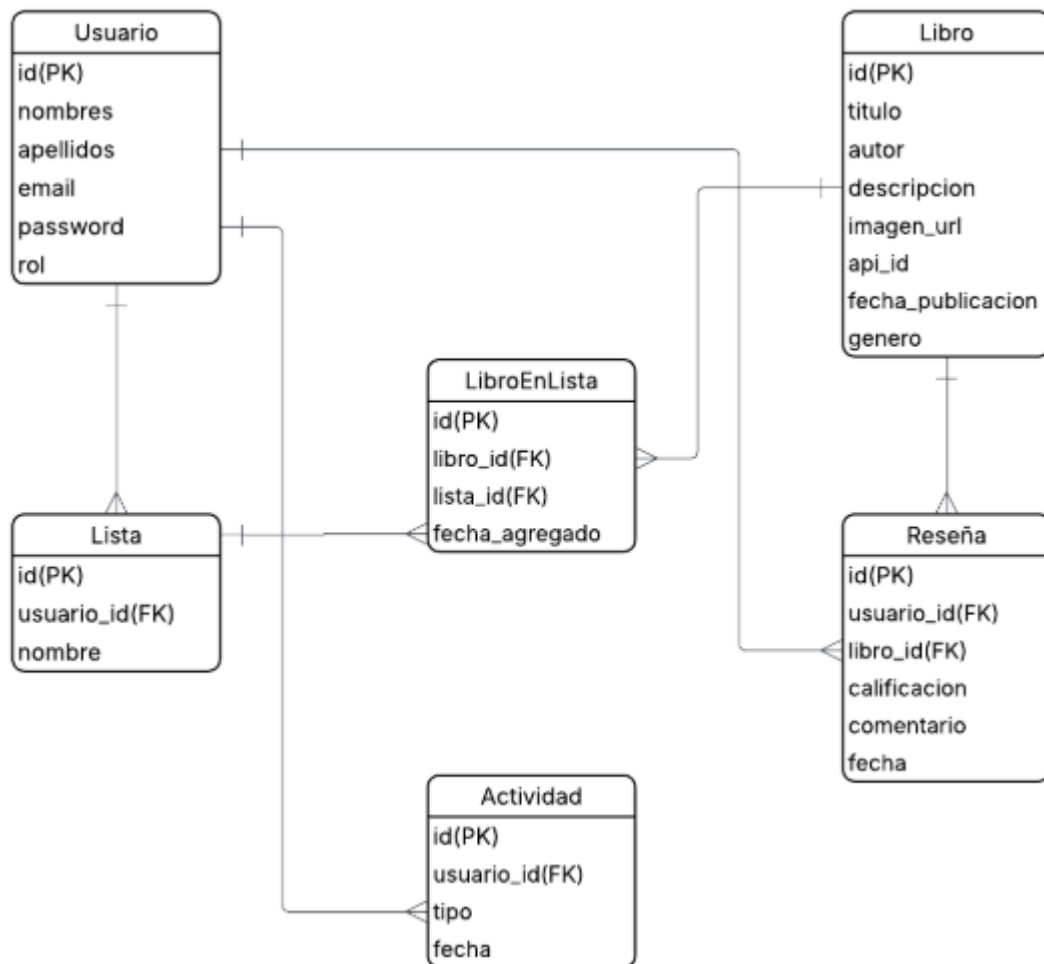
Las relaciones implementadas permiten mantener la coherencia de los datos y evitar redundancia de información.

- Un usuario puede tener múltiples listas.

- Un libro puede pertenecer a varias listas.
- Un usuario puede registrar múltiples reseñas.
- Cada reseña pertenece a un único usuario y un único libro.
- Cada actividad pertenece a un único usuario.

Figura 1

Diagrama de la Base de Datos



Gerardo R. (2026). Grafica representativa de las relaciones entre tablas [Lucidchart].

Proyecto de clase, UTP

7.3.Uso de herramienta Git y Github

Git fue utilizado como sistema de control de versiones distribuido, mientras que GitHub funcionó como plataforma colaborativa para la gestión del proyecto.

Estas herramientas permitieron mantener un historial organizado de cambios, desarrollar funcionalidades en paralelo y coordinar el trabajo del equipo.

7.3.1. Visual Studio Code y otras herramientas

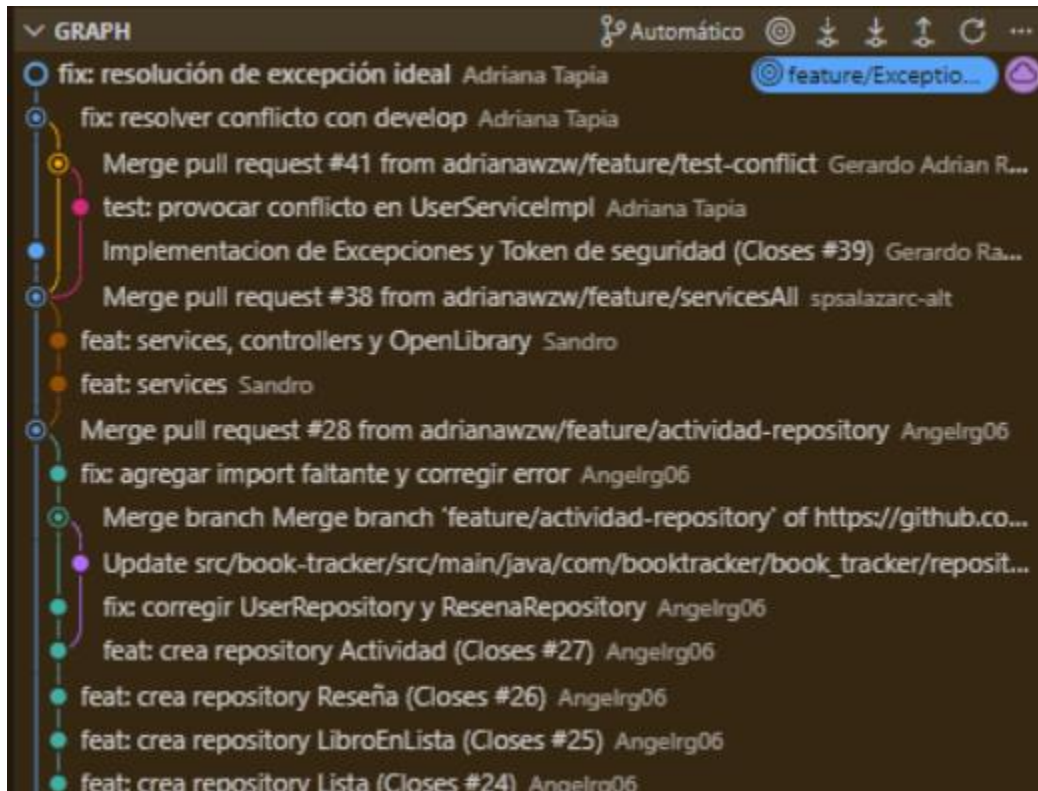
A lo largo de este documento se presentarán y dará uso a diferentes herramientas con el propósito de mejorar el desarrollo del proyecto.

Visual Studio Code fue utilizado como herramienta de apoyo para la resolución de conflictos de integración detectados por Git. Gracias a sus funcionalidades de comparación visual y selección de cambios, permitió identificar las diferencias entre versiones de un mismo archivo, elegir los cambios correctos y completar el proceso de fusión de manera eficiente, reduciendo errores durante la integración del código fuente.

Si se está trabajando con Git y GitHub dentro de VS Code, varias extensiones complementan el control de versiones y facilitan el desarrollo colaborativo. Se usó Git Graph, herramienta que permite visualizar gráficamente el historial de Git, fue utilizada para visualizar el historial de cambios y la estructura de ramas del proyecto, facilitando el seguimiento de los commits y los procesos de integración entre ramas.

Figura 2

Flujo de ramas en Git Graph



Nota. Elaboración propia

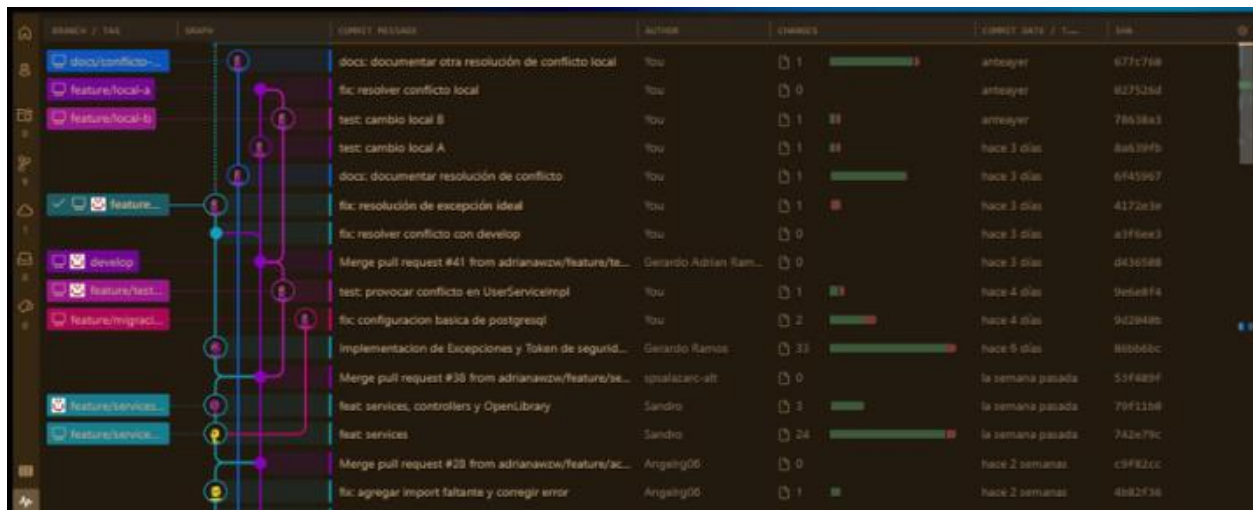
Por otro lado, GitLens amplía las funcionalidades de Git dentro de VS Code. GitLens permitió rastrear la autoría de los cambios realizados en el código fuente, facilitando la revisión de contribuciones y el análisis histórico de modificaciones dentro del repositorio.

Asimismo, a diferencia de Git Graph y GitLens, ESLint no es una herramienta de Git. Es una tecnología de análisis estático para JavaScript y TypeScript. ESLint fue utilizado para verificar la calidad y consistencia del código fuente antes de su integración al repositorio, permitiendo detectar errores y aplicar buenas prácticas de programación.

Por último, GitKraken es un cliente gráfico de Git que permite gestionar repositorios sin depender tanto de comandos en la terminal. Se utiliza para trabajar con Git y GitHub, así como con otros servicios como GitLab y Bitbucket.

Figura 3

Flujo de ramas en Git Kraken



Nota. Elaboración propia

7.3.2. Ciclo de Trabajo y Estados

El flujo de trabajo del proyecto se desarrolló utilizando las principales etapas de Git.

- **Staging:** Es el "área de preparación". Antes de guardar un cambio permanentemente, se seleccionan los archivos modificados que formarán parte de la próxima versión. Permite un control granular sobre qué se incluye y qué no.
- **Commit:** Es una captura o "fotografía" del estado de los archivos en el área de *staging*. Cada commit tiene un identificador único (hash) y un mensaje descriptivo, creando un historial inmutable del progreso del proyecto.

- **Push:** Es la acción de enviar los commits realizados en el repositorio local hacia el repositorio remoto en GitHub. Con esto, los cambios están disponibles para el resto del equipo.

Figura 4

Lista de commits iniciales

Commits on Apr 20, 2026		
Merge pull request #6 from adrianawzw/feature/objetivos	Verified	3a6fb5c
Gerardoarvl authored last week		
docs: redacta objetivos (Closes #2)		b0d7f07
Gerardoarvl committed last week		
Merge pull request #5 from adrianawzw/feature/justificacion	Verified	742ff2b
adrianawzw authored last week		
docs: redacta justificacion (Closes #1)		ecedb5b
adrianawzw committed last week		
Commits on Apr 11, 2026		
docs: agrega archivos iniciales		5f94907
adrianawzw committed 2 weeks ago		
docs: estructura base del proyecto		e29c425
adrianawzw committed 2 weeks ago		
Initial commit	Verified	90050ad
adrianawzw authored 2 weeks ago		

Nota. Elaboración propia

7.3.3. Gestión de ramas y colaboración

Para permitir el desarrollo paralelo, el equipo utilizó múltiples ramas de trabajo. Las ramas principales utilizadas fueron:

- main
- develop
- feature/serviceAll

- entre otras

Asimismo, se empleó conceptos para gestionar las ramas:

- **Branching (Ramificación):** Es la capacidad de desviar la línea principal de desarrollo para trabajar en funcionalidades o correcciones de forma aislada. Esto permite que el código estable en la rama main no se vea afectado hasta que la nueva tarea esté terminada.

Figura 5

Ramas activas

Default	
Branch	Updated
main  	 last month
Your branches	
Branch	Updated
develop  	 yesterday
feature/Exception_JWT 	 5 days ago
Active branches	
Branch	Updated
develop  	 yesterday
feature/Exception_JWT 	 5 days ago
feature/servicesAll 	 2 weeks ago

Nota. Elaboración propia

- **Pull Request (PR):** Es una solicitud formal para integrar los cambios de una rama a otra (generalmente hacia main o develop), esta debe tener una descripción clara de lo que se hizo. En el PR, el equipo revisa el código, propone mejoras y discute los cambios antes de aceptarlos.

Figura 6

Pull request descriptivo

feat: Integracion de services, controllers and OpenLibrary #38 Code

Merged spsalazarc-alt merged 2 commits into develop from feature/servicesAll last week

Conversation 2 Commits 2 Checks 0 Files changed 24 +1,059 -94

Descripción

Se implementó la capa de servicios y controladores para las entidades de este proyecto.

Cambios realizados

- Creación de interfaces Service y clases ServiceImpl.
- Implementación de lógica CRUD para:
 - Usuario
 - Libro
 - Lista
 - LibroEnLista
 - Reseña
 - Actividad
- Creación de Controllers REST para exponer los endpoints de la API.
- Integración de Swagger/OpenAPI para documentación automática de endpoints.
- Integración con la API externa OpenLibrary para búsqueda de libros.
- Configuración de pruebas mediante Swagger UI.
- Actualización del pom.xml con dependencias necesarias.

Resultado parcial

Ahora el proyecto cuenta con endpoints funcionales y documentados automáticamente, permitiendo realizar pruebas CRUD y consultas a la API externa OpenLibrary.

Reviewers

- adrianawzw ✓
- Gerardoarvl ✓
- Angelrg06 ●

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

[Unsubscribe](#)

You're receiving notifications because your review was requested.

Nota. Elaboración propia

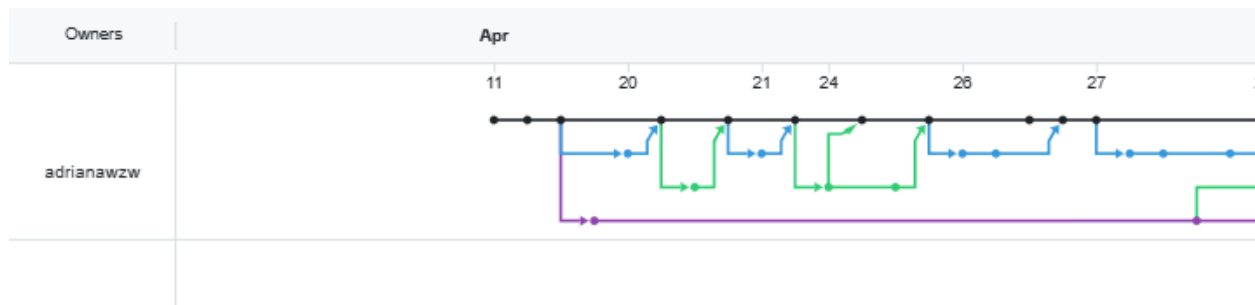
- **Merge:** Es el proceso de fusionar dos ramas. Una vez que un PR es aprobado (en nuestro caso por al menos por 2 personas por el ruleset), se realiza el *merge* para integrar las nuevas funcionalidades al código base del proyecto.

Cada funcionalidad fue desarrollada de manera aislada en una rama feature/, evitando afectar la estabilidad del proyecto principal.

Una vez completadas las tareas, se realizaron Pull Requests para revisar e integrar los cambios mediante procesos de merge.

Figura 7

Flujo de trabajo en GitHub



Nota. Elaboración propia

Figura 8

Pull request cerrada e integrada

feat: Integracion de services, controllers and OpenLibrary #38 Code

Merged spsalazarc-alt merged 2 commits into develop from feature/servicesAll 5 days ago

Conversation 2 Commits 2 Checks 0 Files changed 24 +1,059 -94

spsalazarc-alt commented last week Collaborator

Descripción

Se implementó la capa de servicios y controladores para las entidades de este proyecto.

Cambios realizados

- Creación de interfaces Service y clases ServiceImpl.
- Implementación de lógica CRUD para:
 - Usuario
 - Libro
 - Lista
 - LibroEnLista
 - Reseña
 - Actividad
- Creación de Controllers REST para exponer los endpoints de la API.
- Integración de Swagger/OpenAPI para documentación automática de endpoints.
- Integración con la API externa OpenLibrary para búsqueda de libros.
- Configuración de pruebas mediante Swagger UI.
- Actualización del pom.xml con dependencias necesarias.

Resultado parcial

Ahora el proyecto cuenta con endpoints funcionales y documentados automáticamente, permitiendo realizar pruebas CRUD

Reviewers

- adrianawzw ✓
- Gerardoarvl ✓
- Angelrg06

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

[Customize](#)

Nota. Elaboración propia

7.3.4. Gestión Ágil con GitHub Projects e Issues

Se utilizó GitHub Projects como tablero Kanban para organizar el avance del proyecto. El flujo de trabajo se dividió en columnas:

- To do
- In Progress
- Done

Asimismo, se crearon GitHub Issues para registrar tareas, errores y funcionalidades pendientes. Cada Issue fue asociado a una rama feature/ específica y cerrado automáticamente desde el Pull Request correspondiente.

Figura 9

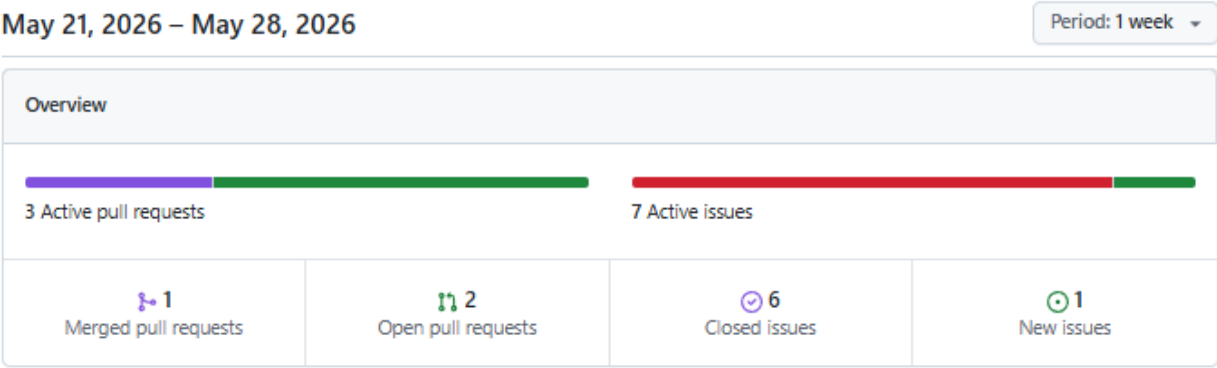
Lista de Issues

☐	Open	1	Closed	22		Author ▾	Labels ▾	Projects ▾	Milestones ▾	Assignees ▾	⇅ ↓ Newest ▾
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										

Nota. Elaboración propia

Figura 10

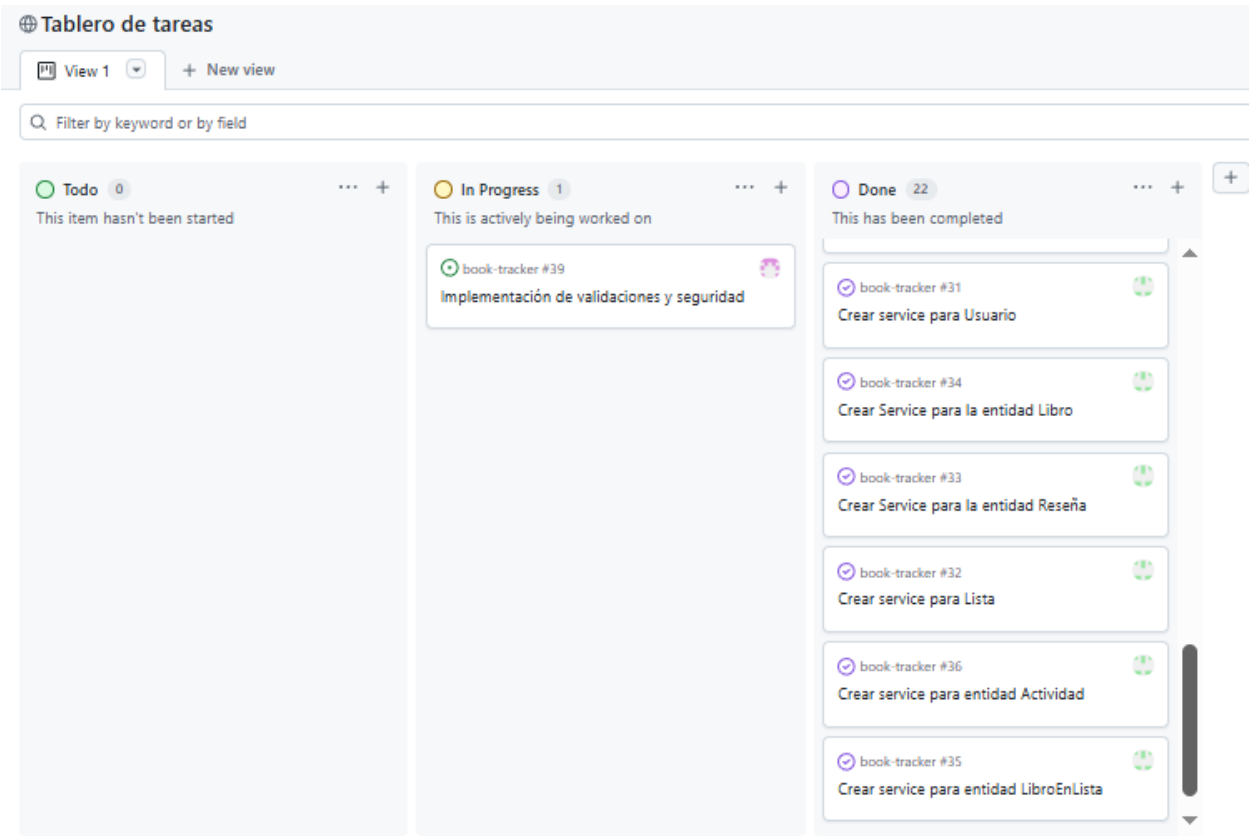
Resumen de Issues y Pull Requests en la última semana



Nota. Elaboración propia

Figura 11

Tablero Kanban con issues cerradas



Nota. Elaboración propia

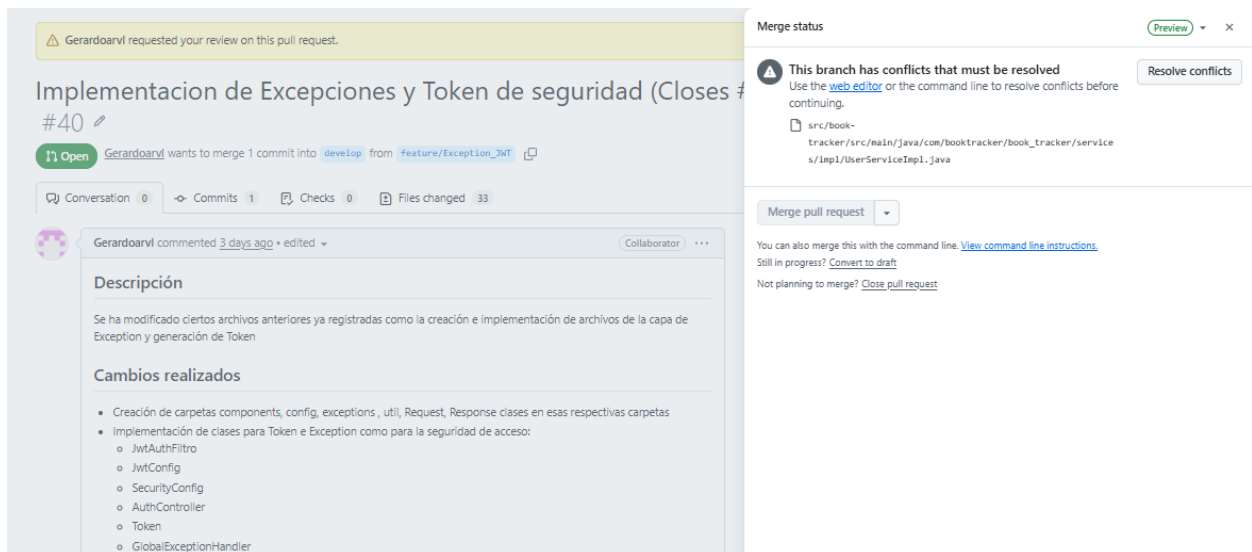
7.3.5. Resolución de conflictos

Durante el desarrollo colaborativo del proyecto se simuló un conflicto de integración utilizando diferentes ramas feature basadas en develop. El conflicto fue provocado modificando simultáneamente la misma sección del archivo UserServiceImpl.java desde dos ramas distintas feature/Exception_JWT con PR abierto y feature/test-conflict.

Inicialmente, GitHub detectó el conflicto en un Pull Request al intentar integrar la rama feature/Exception_JWT hacia develop, mostrando el mensaje de incompatibilidad entre cambios.

Figura 12

Conflicto para el merge hacia develop



Nota. Elaboración propia

Posteriormente, la resolución se realizó localmente utilizando el comando git merge develop desde la rama afectada. La resolución fue realizada manualmente utilizando Visual

Studio Code, donde se visualizaron las marcas de conflicto generadas automáticamente por Git (<<<<<<, =====, >>>>>>). Se seleccionaron los cambios correctos, se eliminaron las marcas y finalmente se confirmó la resolución mediante un commit.

Figura 13

Inicio de resolución

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (develop)
$ git checkout feature/Exception_JWT
Switched to branch 'feature/Exception_JWT'
Your branch is up to date with 'origin/feature/Exception_JWT'.

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT)
$ git merge develop
Auto-merging src/book-tracker/src/main/java/com/booktracker/book_tracker/services/impl/UserServiceImpl.java
CONFLICT (content): Merge conflict in src/book-tracker/src/main/java/com/booktracker/book_tracker/services/impl/UserServiceImpl.java
Automatic merge failed; fix conflicts and then commit the result.

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT|MERGING)
$ |
```

Nota. Elaboración propia

Figura 14

Marcas de conflicto en Visual Studio Code

```
src > main > java > com > booktracker > book_tracker > services > impl > UserServiceImpl.java > Language Support for Java(TM) by Red Hat > UserServiceImpl.java
23 public class UserServiceImpl implements UserService, UserDetailsService {
33     public List<UserResponseDTO> obtenerTodos() {
34         return userRepository.findAll()
35             .stream()
36             .map(UserMapper::toDTO)
37             .toList();
38     }
39
40     @Override
41     public UserResponseDTO obtenerPorId(Long id) {
42
43
44         User usuario = userRepository.findById(id)
45         <<<<<< HEAD (Cambio actual)
46         .orElseThrow(() -> new ResourceNotFoundException("Usuario no encontrado"));
47         =====
48
49         .orElseThrow(() -> new IllegalStateException("Usuario inválido"));
50         >>>>>> develop (Cambio entrante)
51
52         return UserMapper.toDTO(usuario);
53     }
54
55     @Override
56     public UserResponseDTO crearUsuario(UserRequestDTO dto) {
57
```

Nota. Elaboración propia

El proceso completo, incluyendo comandos utilizados, ramas involucradas y capturas del conflicto, fue documentado en el archivo Markdown correspondiente.

Figura 15

Conflicto solucionado


```

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT|MERGING)
$ git status
On branch feature/Exception_JWT
Your branch is up to date with 'origin/feature/Exception_JWT'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   src/main/java/com/booktracker/book_tracker/services/impl/UserServiceImpl.java

no changes added to commit (use "git add" and/or "git commit -a")

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT|MERGING)
$ git add .

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT|MERGING)
$ git commit -m "fix: resolver conflicto con develop"
[feature/Exception_JWT a3f6ee3] fix: resolver conflicto con develop

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker/src/book-tracker (feature/Exception_JWT)
$ git push origin feature/Exception_JWT
Enumerating objects: 37, done.
Counting objects: 100% (37/37), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 1005 bytes | 201.00 KiB/s, done.
Total 13 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/adrianawzw/book-tracker.git
   86bb6bc..a3f6ee3  feature/Exception_JWT -> feature/Exception_JWT

```

Nota. Elaboración propia

Figura 16

Conflicto solucionado y esperando aprobación para merge

Gerardoarvl requested your review on this pull request. [Add your review](#)

Implementacion de Excepciones y Token de seguridad (Closes #39)

[Awaiting approval](#) [Code](#)

#40 [Open](#) Gerardoarvl wants to merge 2 commits into `develop` from `feature/Exception_JWT`

Conversation 0 Commits 2 Checks 0 Files changed 33 +891 -74

Gerardoarvl commented 3 days ago • edited Collaborator

Descripción

Se ha modificado ciertos archivos anteriores ya registradas como la creación e implementación de archivos de la capa de Exception y generación de Token

Cambios realizados

- Creación de carpetas components, config, exceptions, util, Request, Response clases en esas respectivas carpetas
- Implementación de clases para Token e Exception como para la seguridad de acceso:
 - JwtAuthFiltro
 - JwtConfig

Reviewers

- Angelrg06
- adrianawzw
- spsalazarc-alt

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Nota. Elaboración propia

El proceso completo, incluyendo comandos utilizados, ramas involucradas y capturas del conflicto, fue documentado en el archivo Markdown correspondiente.

Asimismo, el conflicto mencionado y resuelto fue uno detectado remotamente en el PR de GitHub, pero la resolución se hizo localmente usando Git y VS Code. Por ende, también se hizo la simulación de un conflicto local que también fue abarcado en el archivo Markdown.

7.3.6. Control de versiones y tags

Se utilizaron tags para identificar versiones importantes del proyecto y mantener un control organizado del avance del sistema.

Entre las versiones registradas se encuentra:

- v1.0 → Primera versión funcional

Los tags fueron creados utilizando comandos Git como:

```
git tag -a v1.0. -m ""
```

Figura 17

Tag creado

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (develop)
$ git tag -a v1.0 -m "Versión final de AVTF2 del proyecto"

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (develop)
$ git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 191 bytes | 191.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/adrianawzw/book-tracker.git
 * [new tag]          v1.0 -> v1.0

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (develop)
```

Nota. Elaboración propia

Un Release es una versión publicada oficialmente del proyecto. En GitHub, normalmente un Release se crea a partir de un Tag, este puede incluir: nombre de la versión, descripción, cambios realizados, archivos descargables.

Figura 18

Primer release



Nota. Elaboración propia

7.3.7. Revisiones, sugerencias y comentarios

Cuando alguien revisa el código del PR puede aprobarlo, pedir cambios y dejar comentarios. Los comentarios son mensajes que se dejan sobre líneas específicas del código que sirven para detectar errores, mejorar código, sugerir buenas prácticas o pedir cambios. Por otro lado, GitHub permite dejar cambios sugeridos directamente por lo que puede aparecer como sugerencia para que el autor simplemente haga clic y GitHub aplique el cambio automáticamente.

Figura 19

Sugerencia de código aceptada y aplicada

Merged Feature/actividad repository #28
Angelrg06 merged 10 commits into `develop` from `feature/actividad-repository`

Angelrg06 requested a review from [adrianawzw](#) 2 weeks ago

[adrianawzw](#) reviewed 2 weeks ago [View reviewed changes](#)

src/book-tracker/src/main/java/com/booktracker/book_tracker/repositories/LibroRepository.java

Outdated Hide resolved

```
8 +
9 + @Repository
10 + public interface LibroRepository extends JpaRepository<Libro, Long> {
11 +     Libro findByIsbn(String isbn);
```

[adrianawzw](#) 2 weeks ago Owner ...

Suggested change

-	Libro findByIsbn(String isbn);
+	Optional<Libro> findById(String apiId);

This suggestion has been applied or marked resolved

Commit suggestion

Reply...

[Unresolve conversation](#) Angelrg06 marked this conversation as resolved.

Nota. Elaboración propia

Figura 20

Comentario de recomendación

Merged

Feature/actividad repository #28

Angelrg06 merged 10 commits into develop from feature/actividad-repository



adrianawzw reviewed 2 weeks ago

View reviewed changes

src/book-tracker/src/main/java/com/booktracker/book_tracker/repositories/ResenaRepository.java

Outdated

10 9

11 10 @Repository

12 - public interface ResenaRepository extends JpaRepository<Resena, Long>{

11 + public interface ReseñaRepository extends JpaRepository<Resena, Long>{

 adrianawzw 2 weeks ago

Owner ...

La clase ReseñaRepository debería ser ResenaRepository para evitar errores en la compilación y encoding



Reply...

Resolve conversation



adrianawzw reviewed 2 weeks ago

View reviewed changes

src/book-tracker/src/main/java/com/booktracker/book_tracker/repositories/UsuarioRepository.java

Outdated

10 + User findByUsername(String username);

11 +

12 + boolean existsByEmail(String email);

13 + boolean existsByUsername(String username);

 adrianawzw 2 weeks ago

Owner ...

El modelo de User no tiene username, tiene email por lo que los métodos con username no deberían estar ahí

Nota. Elaboración propia

Figura 21

Cambios realizados y aprobación

The screenshot displays a GitHub Pull Request interface. At the top, a commit history shows four commits: 'Update src/book-tracker/src/main/java/com/booktracker/book_tracker/repositories/LibroRepository.java' (Verified, bd334d5), 'fix: corregir UserRepository y ResenaRepository' (8a61354), 'Merge branch Merge branch 'feature/actividad-repository' of https://g...' (eb91458), and 'fix: agregar import faltante y corregir error' (4b82f36). Below the history, a review by 'adrianawzw' is shown, indicating approval '2 weeks ago' with a 'View reviewed changes' link. A comment box shows 'adrianawzw' left a comment: 'Corregido las sugerencias adecuadamente'. Below the comment, 'Angelrg06' is shown merging commit 'c9f82cc' into the 'develop' branch '2 weeks ago', with a 'Revert' button. At the bottom, a notification states 'adrianawzw deleted the feature/actividad-repository branch last week' with a 'Restore branch' button. A final purple notification box at the bottom reads 'Pull request successfully merged and closed' and 'You're all set — the branch has been merged.'

Nota. Elaboración propia

7.3.8. Integración Continua con GitHub Actions

La Integración Continua (Continuous Integration, CI) es una práctica de desarrollo de software que consiste en integrar de manera frecuente los cambios realizados por los desarrolladores en un repositorio compartido. Su principal objetivo es detectar errores de forma temprana mediante la ejecución automática de procesos de compilación y pruebas.

Para el proyecto Book Tracker se implementó un flujo de Integración Continua utilizando GitHub Actions. Esta herramienta permitió automatizar la validación del proyecto cada vez que se realiza un push sobre la rama main o un Pull Request dirigido a las ramas develop y main.

Para ello, se configuró un archivo de workflow dentro del directorio `.github/workflows`, encargado de ejecutar automáticamente las diferentes etapas del proceso.

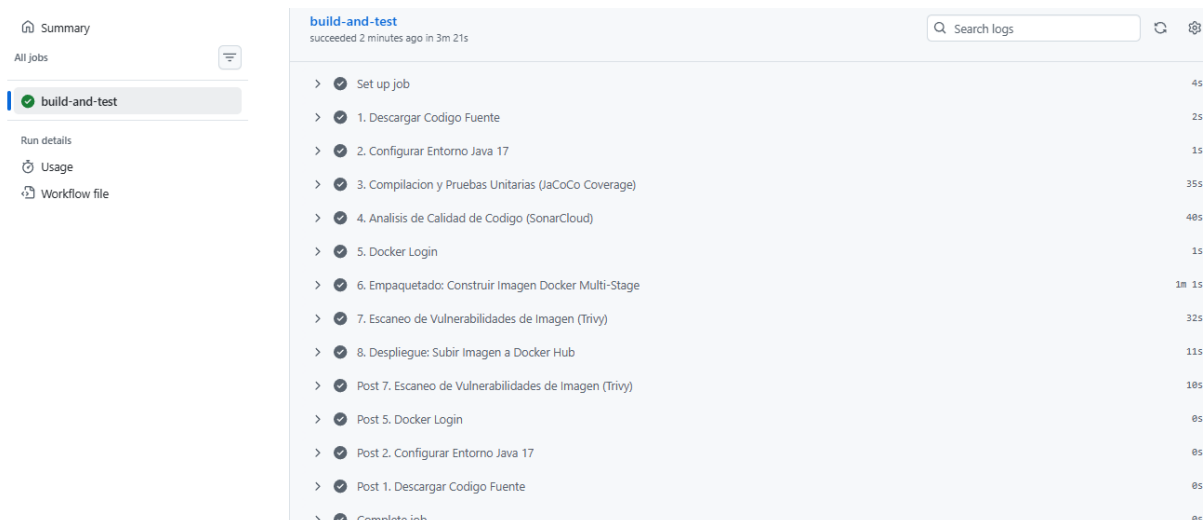
El flujo configurado incluye la instalación de Java 17, la compilación del proyecto mediante Maven y la ejecución del comando `mvn clean verify`, permitiendo validar el correcto funcionamiento del backend desarrollado con Spring Boot. De esta manera, cada modificación realizada en el código es verificada antes de ser integrada al proyecto, garantizando una mayor estabilidad y reduciendo la posibilidad de introducir errores en versiones futuras.

Además de la compilación y las pruebas automatizadas, el workflow incorpora el análisis de calidad con SonarCloud, la construcción de una imagen Docker Multi-Stage, el escaneo de vulnerabilidades mediante Trivy y la publicación de la imagen generada en Docker Hub, facilitando posteriormente su despliegue en Azure.

La implementación de GitHub Actions permitió mejorar la calidad del desarrollo colaborativo al proporcionar retroalimentación inmediata sobre el estado de la aplicación y asegurar que las funcionalidades implementadas cumplan con los requisitos establecidos.

Figura 22

Detalle del job del workflow



Nota. Elaboración propia

7.3.9. Calidad de código con SonarCloud

Con el objetivo de evaluar la calidad del código fuente y detectar posibles vulnerabilidades, se integró la plataforma SonarCloud al proceso de desarrollo del proyecto. SonarCloud es una herramienta de análisis estático que permite identificar errores de programación, vulnerabilidades de seguridad, duplicación de código y otros aspectos relacionados con las buenas prácticas de desarrollo.

La integración se realizó mediante GitHub Actions, permitiendo que cada ejecución del pipeline genere automáticamente un análisis del código fuente. Entre las métricas evaluadas se encuentran la cantidad de bugs detectados, vulnerabilidades de seguridad, code smells y el cumplimiento de estándares de calidad establecidos por la herramienta.

Gracias a esta integración fue posible identificar oportunidades de mejora en el código del backend, así como verificar que las correcciones implementadas mantuvieran un nivel adecuado de calidad. Asimismo, SonarCloud proporcionó un panel de control centralizado que facilitó el seguimiento de los resultados obtenidos durante el desarrollo del proyecto.

7.3.10. Pipeline CI/CD

Con el objetivo de automatizar el proceso de integración continua y facilitar el despliegue de la aplicación, se implementó un pipeline utilizando GitHub Actions. Este pipeline permite ejecutar automáticamente una serie de tareas de validación cada vez que se realizan cambios en el repositorio, garantizando que el código cumpla con los criterios de calidad establecidos antes de ser distribuido.

El flujo se activa automáticamente cuando se realiza un push sobre la rama principal (main) o cuando se crea o actualiza un Pull Request dirigido a las ramas develop o main. A partir de estos eventos, GitHub Actions ejecuta de forma secuencial las diferentes etapas definidas dentro del workflow.

Las etapas implementadas comprenden la compilación del proyecto y la ejecución de pruebas unitarias mediante Maven, el análisis de calidad del código utilizando SonarCloud, la autenticación con Docker Hub, la construcción de una imagen Docker Multi-Stage, el análisis de vulnerabilidades de la imagen mediante Trivy y, finalmente, la publicación de la imagen generada en Docker Hub.

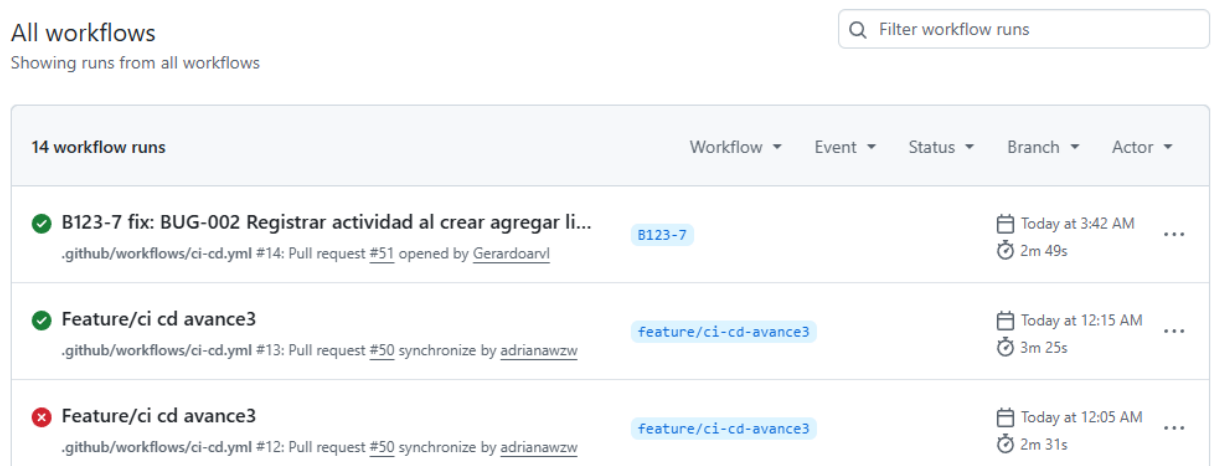
Una vez publicada la imagen, el despliegue del backend se realiza de forma automática mediante la integración entre Docker Hub y Azure App Service, el cual detecta la nueva versión de la imagen mediante un Webhook y actualiza el contenedor desplegado sin necesidad de ejecutar tareas adicionales desde GitHub Actions.

La implementación de este pipeline permitió automatizar el proceso de integración continua, reducir las tareas manuales durante la construcción y validación del proyecto, asegurar la calidad y seguridad del software antes de su publicación y mantener sincronizada la aplicación

desplegada con las versiones más recientes almacenadas en Docker Hub. El workflow completo implementado se presenta en la sección de anexos.

Figura 23

Historial de ejecuciones del workflow



All workflows		Filter workflow runs	
Showing runs from all workflows			
14 workflow runs		Workflow ▾	Event ▾
		Status ▾	Branch ▾
		Actor ▾	
✓	B123-7 fix: BUG-002 Registrar actividad al crear agregar li... .github/workflows/ci-cd.yml #14: Pull request #51 opened by Gerardoanvl	B123-7	Today at 3:42 AM 2m 49s
✓	Feature/ci cd avance3 .github/workflows/ci-cd.yml #13: Pull request #50 synchronize by adrianawzw	feature/ci-cd-avance3	Today at 12:15 AM 3m 25s
✗	Feature/ci cd avance3 .github/workflows/ci-cd.yml #12: Pull request #50 synchronize by adrianawzw	feature/ci-cd-avance3	Today at 12:05 AM 2m 31s

Nota. Elaboración propia

7.3.11. Dockerización de la aplicación

Con el propósito de facilitar la distribución y ejecución del sistema en diferentes entornos, se realizó la dockerización de la aplicación mediante el uso de contenedores Docker. Docker es una plataforma que permite empaquetar aplicaciones junto con todas sus dependencias, garantizando un comportamiento consistente independientemente del entorno donde se ejecuten.

Para ello se creó un archivo Dockerfile que contiene las instrucciones necesarias para construir una imagen de la aplicación Book Tracker. Dicho archivo define el entorno de

ejecución, copia los archivos requeridos y configura el inicio automático del servicio desarrollado con Spring Boot.

La utilización de Docker proporciona ventajas importantes, entre las que destacan la portabilidad, la facilidad de despliegue y la reducción de problemas relacionados con diferencias de configuración entre equipos de desarrollo y entornos de producción. Asimismo, permitió ejecutar la aplicación de manera aislada mediante contenedores, simplificando las tareas de prueba y distribución.

Figura 24

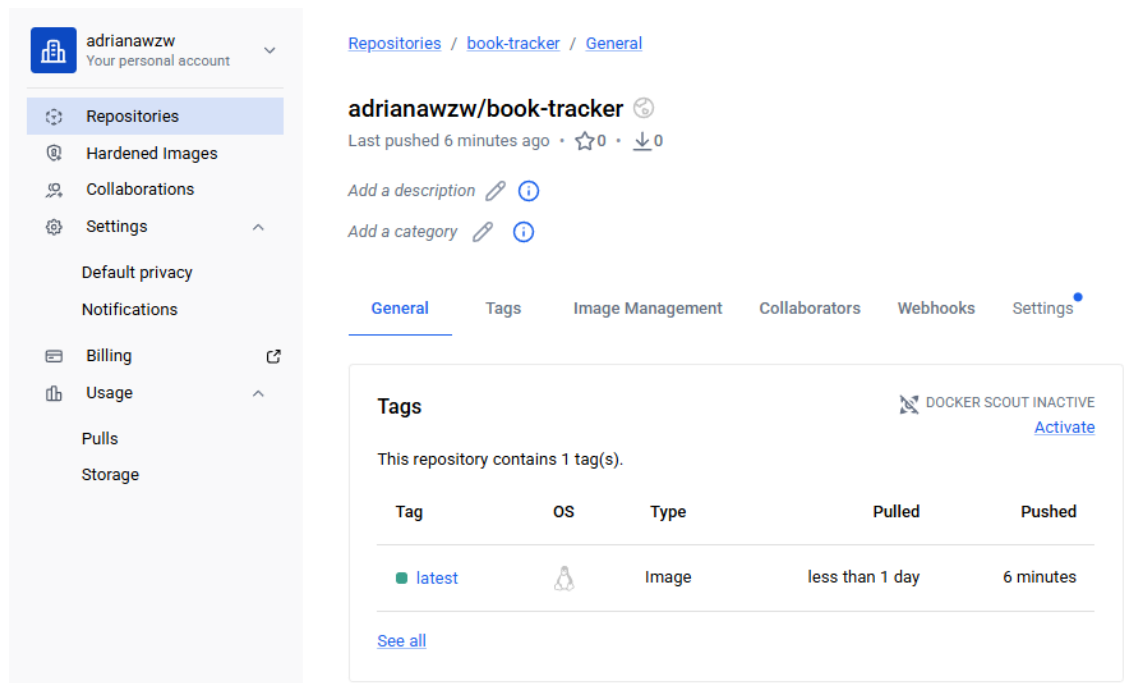
Imagen del Docker

```
=> => unpacking to docker.io/library/book-tracker:latest  
  
D:\Proyectos\Proyecto-HerramientasDeDesarrollo\book-tracker\src\book-tracker>docker images  
REPOSITORY      TAG         IMAGE ID      CREATED        SIZE  
book-tracker     latest      236a5e2645cf  About a minute ago  549MB  
  
D:\Proyectos\Proyecto-HerramientasDeDesarrollo\book-tracker\src\book-tracker>
```

Nota. Elaboración propia

Figura 25

Imagen del publicada en DockerHub



Nota. Elaboración propia

7.3.12. Docker Multi-Stage

Como parte de las buenas prácticas de contenedorización, se implementó un proceso de construcción Multi-Stage en el Dockerfile de la aplicación. Esta técnica consiste en dividir la construcción de la imagen en múltiples etapas, cada una con una función específica.

La primera etapa se encargó de compilar el proyecto utilizando Maven y generar el archivo ejecutable correspondiente. Posteriormente, una segunda etapa utilizó una imagen más ligera basada únicamente en Java Runtime Environment (JRE), copiando únicamente el archivo generado durante la compilación.

La principal ventaja de esta estrategia es la reducción significativa del tamaño de la imagen final, ya que evita incluir herramientas de desarrollo y dependencias innecesarias en el

entorno de ejecución. Como resultado, se obtuvo una imagen más eficiente, segura y adecuada para procesos de despliegue automatizados.

Figura 26

Archivo Dockerfile

```
# =====  
# ETAPA 1: BUILD  
# =====  
FROM maven:3.9.9-eclipse-temurin-17 AS builder  
  
WORKDIR /app  
  
COPY pom.xml .  
  
RUN mvn dependency:go-offline  
  
COPY src ./src  
  
RUN mvn clean package -DskipTests  
# =====  
# ETAPA 2: RUNTIME  
# =====  
FROM eclipse-temurin:17-jre  
  
WORKDIR /app  
  
COPY --from=builder /app/target/*.jar app.jar  
  
EXPOSE 8080  
  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Nota. Elaboración propia

7.3.13. Seguridad de dependencias con Snyk

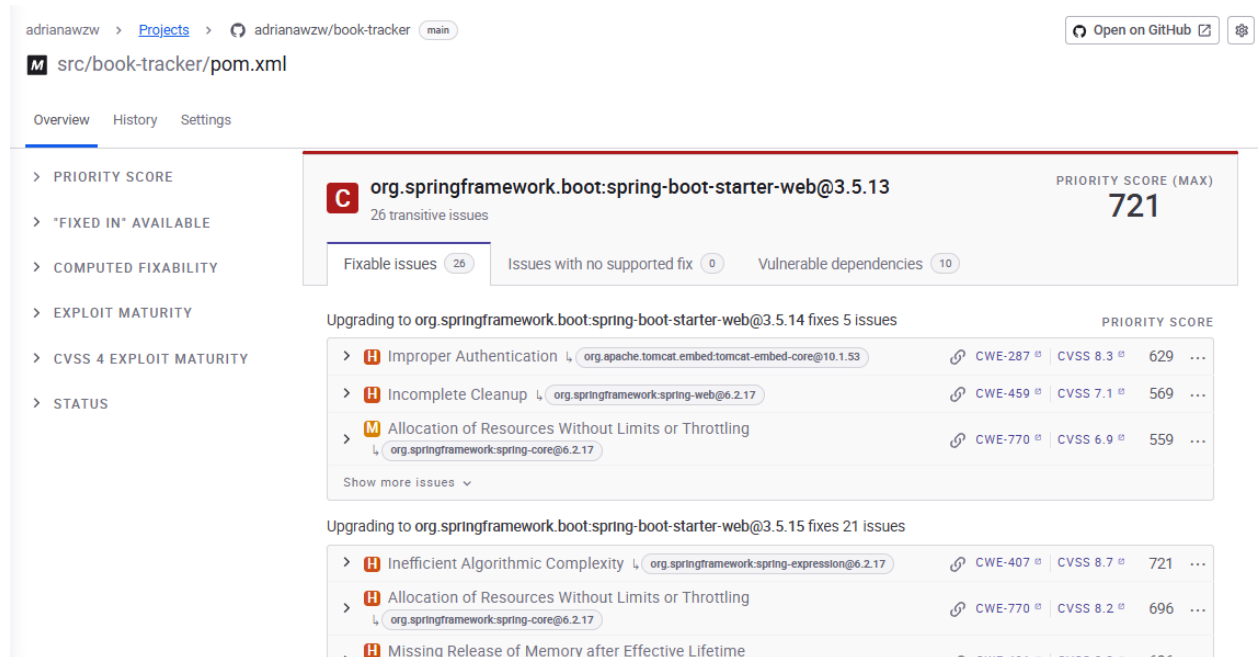
Con el propósito de fortalecer la seguridad del proyecto, se integró la herramienta Snyk para el análisis de dependencias utilizadas por la aplicación. Snyk permite identificar vulnerabilidades conocidas presentes en librerías de terceros, proporcionando información detallada sobre su nivel de criticidad y posibles acciones de remediación.

Durante el desarrollo del proyecto Book Tracker se realizó un escaneo de las dependencias gestionadas mediante Maven, obteniendo un reporte de vulnerabilidades potenciales asociadas a determinadas versiones de librerías utilizadas por el sistema. Esta información permitió identificar componentes que requerían actualización o revisión para reducir riesgos de seguridad.

La utilización de Snyk complementó las actividades de calidad y seguridad implementadas en el pipeline DevSecOps del proyecto, contribuyendo a mejorar la confiabilidad y mantenibilidad de la aplicación.

Figura 27

Snyk issues



Nota. Elaboración propia

7.3.14. Escaneo de vulnerabilidades de imágenes con Trivy

Como parte de las prácticas de DevSecOps implementadas en el proyecto, se utilizó la herramienta Trivy para realizar análisis de seguridad sobre las imágenes Docker generadas durante el proceso de construcción. Trivy es un escáner de vulnerabilidades que permite identificar riesgos asociados a sistemas operativos, librerías y paquetes incluidos dentro de una imagen de contenedor.

Una vez construida la imagen Docker de Book Tracker, el pipeline ejecuta automáticamente un análisis de vulnerabilidades de severidad alta (HIGH) y crítica (CRITICAL). Los resultados obtenidos permiten identificar posibles riesgos de seguridad antes de publicar la imagen en Docker Hub.

La incorporación de Trivy permitió añadir una capa adicional de validación de seguridad al proceso de integración continua, contribuyendo a la detección temprana de vulnerabilidades y fortaleciendo la calidad general de la solución desarrollada.

7.3.15. Despliegue de la aplicación en Azure

7.3.15.1. Arquitectura de despliegue

Con el propósito de poner la aplicación a disposición de los usuarios en un entorno de producción, se realizó el despliegue de todos los componentes principales del sistema utilizando los servicios de Microsoft Azure. La arquitectura implementada está conformada por el frontend desarrollado en Angular, el backend construido con Spring Boot y la base de datos SQL, cada uno alojado en un servicio especializado de la plataforma Azure.

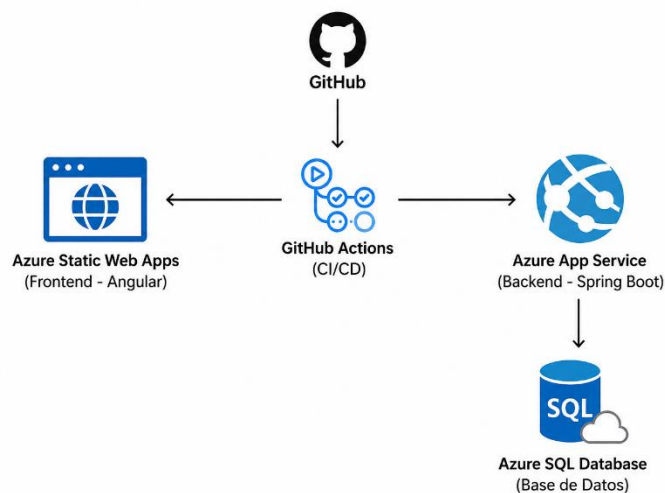
El frontend fue publicado mediante Azure Static Web Apps, el backend fue desplegado en Azure App Service, mientras que la persistencia de la información se encuentra alojada en Azure

PostgreSQL Flexible. La comunicación entre estos componentes se realiza mediante servicios REST, permitiendo que la interfaz de usuario consuma las funcionalidades implementadas en la API y que esta gestione las operaciones sobre la base de datos.

Como resultado, la aplicación puede ejecutarse completamente desde la nube, facilitando su acceso desde cualquier navegador sin necesidad de instalar componentes adicionales.

Figura 28

Arquitectura general del despliegue en Azure



Nota. Elaboración propia

7.3.15.2. Despliegue del Frontend

El frontend de la aplicación, desarrollado con Angular, fue desplegado utilizando Azure Static Web Apps. Este servicio permite alojar aplicaciones web estáticas y mantener una integración directa con repositorios de GitHub.

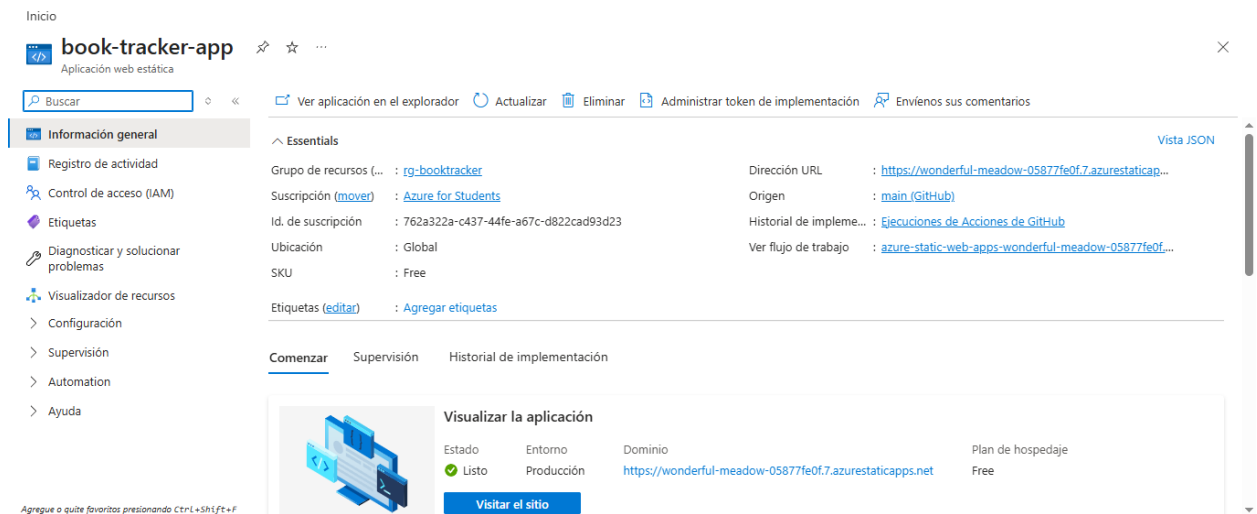
Durante la creación del recurso se seleccionó el repositorio correspondiente y la rama principal del proyecto, permitiendo que Azure generara automáticamente el flujo de integración continua mediante GitHub Actions.

Posteriormente se configuró el secreto AZURE_STATIC_WEB_APPS_API_TOKEN, el cual permite autenticar el proceso de despliegue de forma segura.

Cada vez que se realiza un cambio sobre la rama principal del repositorio, GitHub Actions ejecuta automáticamente la compilación del proyecto Angular y publica la versión actualizada en Azure Static Web Apps.

Figura 29

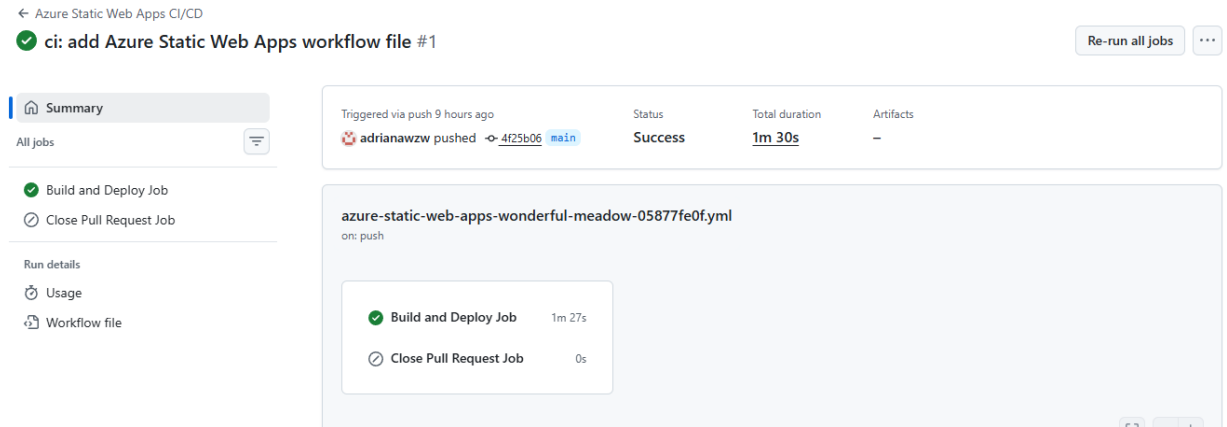
Recurso Azure Static Web Apps



Nota. Elaboración propia

Figura 30

Workflow de GitHub Actions



Nota. Elaboración propia

7.3.15.3. Despliegue del Backend

La API REST desarrollada con Spring Boot fue desplegada utilizando Azure App Service. Este servicio permite alojar aplicaciones Java de manera administrada, proporcionando un entorno preparado para ejecutar servicios web sin necesidad de configurar servidores manualmente.

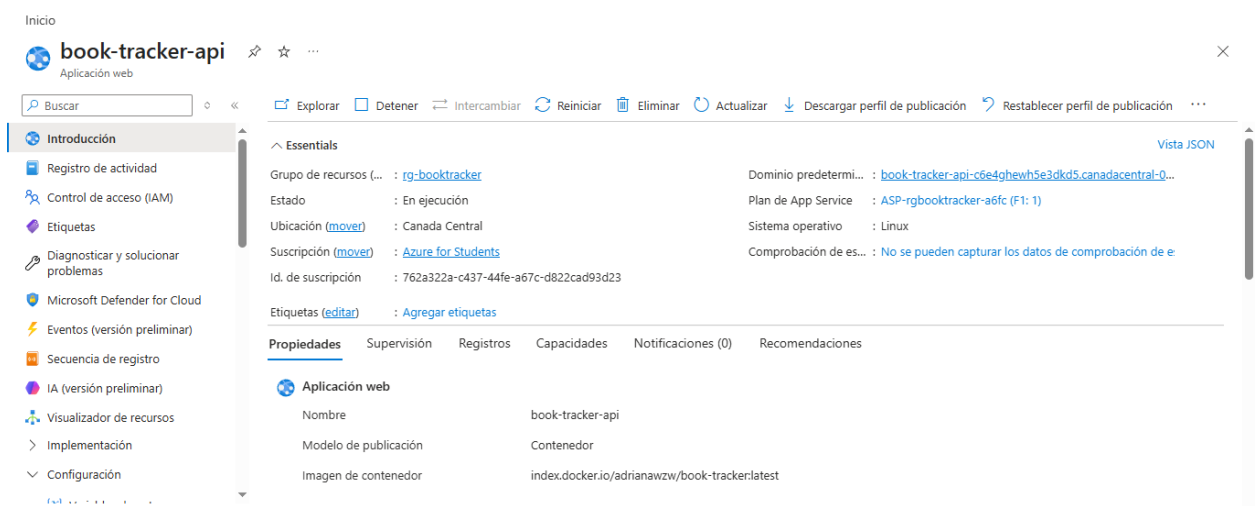
El despliegue se automatizó mediante GitHub Actions. Durante la ejecución del pipeline, el proyecto se compila utilizando Maven, se generan los archivos ejecutables y posteriormente se publican automáticamente en Azure App Service.

Una vez completado el proceso, la API queda disponible mediante una URL pública desde la cual los clientes pueden consumir los diferentes endpoints implementados.

El uso de Azure App Service permite además administrar variables de entorno, configuración de la aplicación y escalabilidad del servicio.

Figura 31

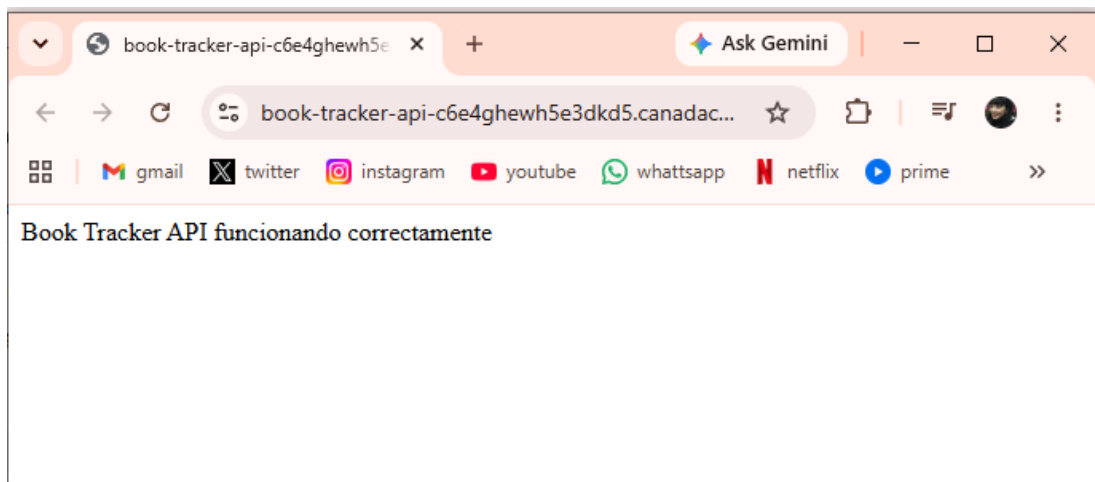
Recurso Azure App Service



Nota. Elaboración propia

Figura 32

API desplegado ejecutándose



Nota. Elaboración propia

7.3.15.4. *Despliegue de la Base de Datos*

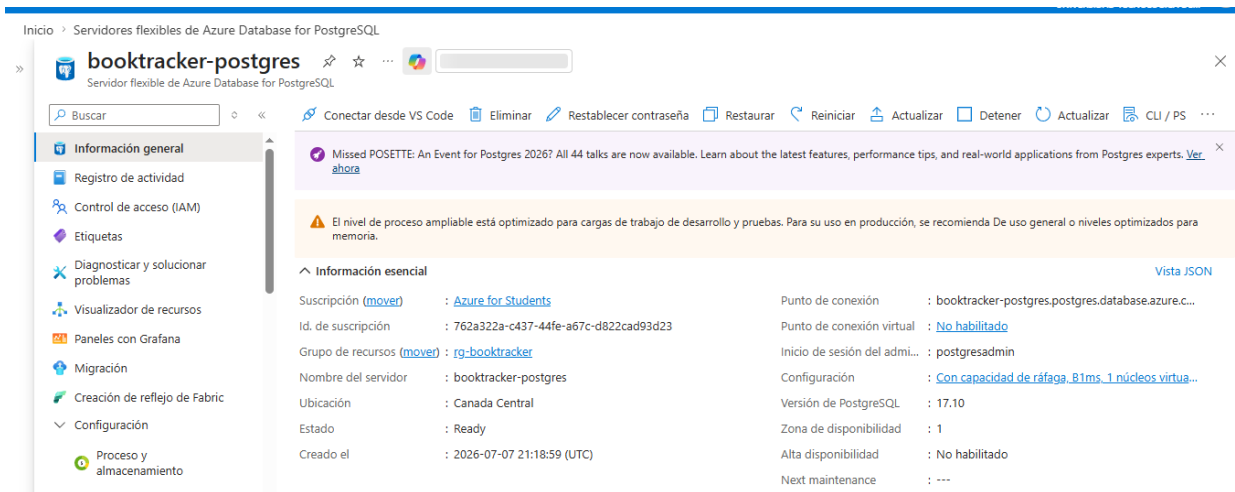
Para el almacenamiento persistente de la información se utilizó Azure Database for PostgreSQL Flexible Server, servicio administrado que ofrece alta disponibilidad, seguridad y respaldo automático de los datos.

Inicialmente se creó el servidor lógico de SQL Database y posteriormente la base de datos correspondiente al proyecto. La conexión entre el backend y la base de datos se configuró mediante la cadena de conexión definida en las variables de entorno del servicio Azure App Service, evitando almacenar credenciales directamente dentro del código fuente.

De esta manera, todas las operaciones realizadas desde la aplicación son almacenadas y consultadas directamente desde la base de datos alojada en Azure.

Figura 33

Azure Database for PostgreSQL Flexible Server



Nota. Elaboración propia

7.3.15.5. Validación del despliegue

Una vez desplegados los componentes principales de la aplicación, se realizaron pruebas para verificar el correcto funcionamiento del sistema en el entorno de producción. Durante estas pruebas se comprobó que el frontend alojado en Azure Static Web Apps consume correctamente los servicios REST proporcionados por el backend desplegado en Azure App Service, el cual mantiene comunicación con la base de datos Azure SQL Database para realizar las operaciones de consulta y actualización de la información.

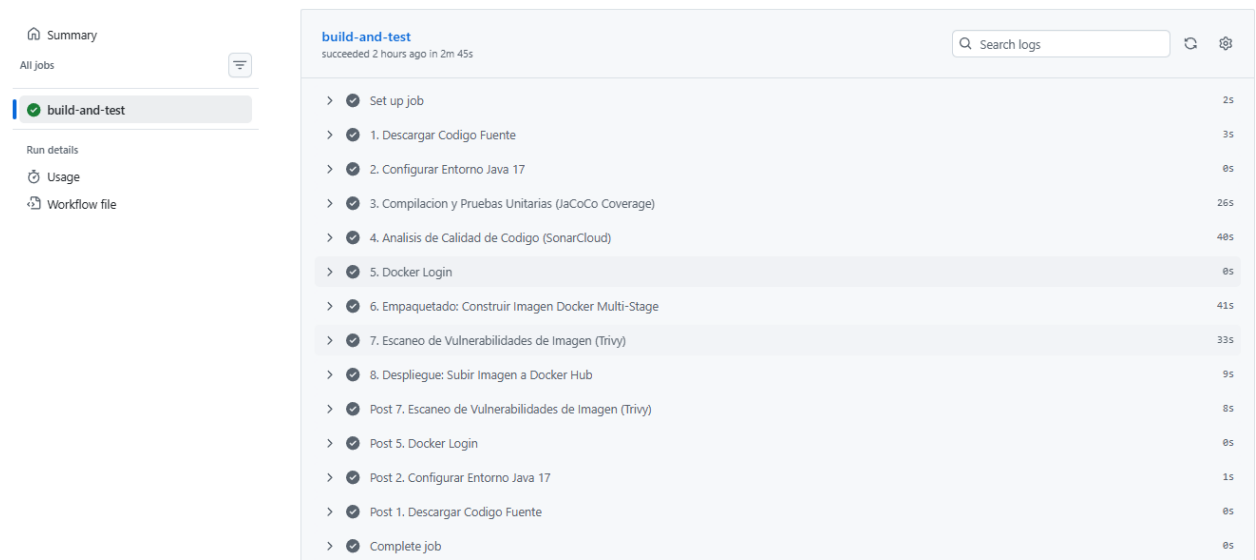
El proceso de integración continua inicia cuando se realiza un cambio en la rama principal del repositorio de GitHub. Automáticamente, GitHub Actions ejecuta la compilación del proyecto, las pruebas unitarias, el análisis de calidad con SonarCloud, el escaneo de vulnerabilidades mediante Trivy y la construcción de la imagen Docker Multi-Stage. Finalmente, la imagen generada es publicada en Docker Hub.

Posteriormente, gracias a la configuración del Webhook entre Docker Hub y Azure App Service, la publicación de una nueva imagen desencadena automáticamente la actualización del contenedor en Azure, permitiendo desplegar la versión más reciente del backend sin necesidad de realizar intervenciones manuales.

Las pruebas realizadas evidenciaron que el proceso completo de integración y despliegue continuo funciona correctamente, asegurando que las nuevas versiones del sistema sean publicadas de manera automática y que todos los componentes permanezcan sincronizados durante su ejecución.

Figura 34

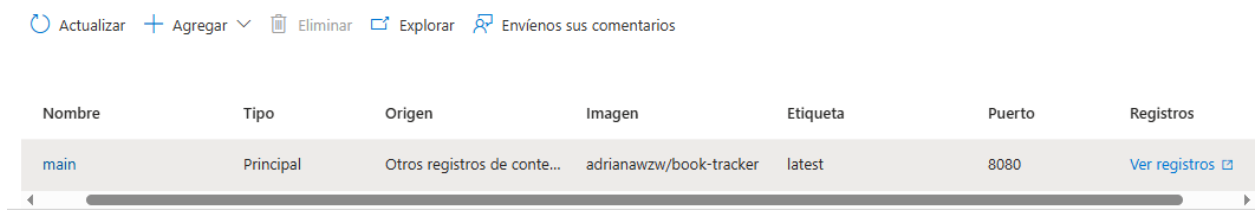
Ejecución satisfactoria del pipeline final en GitHub Actions



Nota. Elaboración propia

Figura 35

Configuración del Webhook entre Docker Hub y Azure App Service



Nota. Elaboración propia

Figura 36

Aplicación consumiendo la API

```
Code Blame 2 lines (2 loc) · 111 Bytes

1 export const API_BASE_URL =
2   'https://book-tracker-api-c6e4ghewh5e3dkd5.canadacentral-01.azurewebsites.net/';
```

Nota. Elaboración propia

7.4. Herramientas de organización y desarrollo de proyectos

Jira y Trello son herramientas de gestión de proyectos y organización del trabajo utilizadas para planificar, coordinar y dar seguimiento a tareas dentro de un equipo. Su objetivo principal es ayudar a controlar tareas existe, quien las realiza, el estado y el avance del proyecto. En el presente proyecto se decidió aplicar Jira la cual es una herramienta enfocada principalmente en el desarrollo de software y metodologías ágiles.

Figura 37

Lista de actividades en JIRA

Book Tracker - AFinal						
Q Buscar actividad  Filtro Agrupar						
☐	Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución
☐	☒ BTA-1 Proyecto Spring boot base	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-2 Repositorio y su estructura	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-3 Justificacion del proyecto	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-4 Objetivos del proyecto	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-5 Marco teorico	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-6 Metodologia	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-7 Documento y ppt	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-8 README.md	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-9 Diagrama de BD en resultados	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-10 Implementacion de entidades	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver
☐	☒ BTA-11 Creacion de repositorios	 Sin asignar	 Samir Rojas	 Medium	 POR HACER	Sin resolver

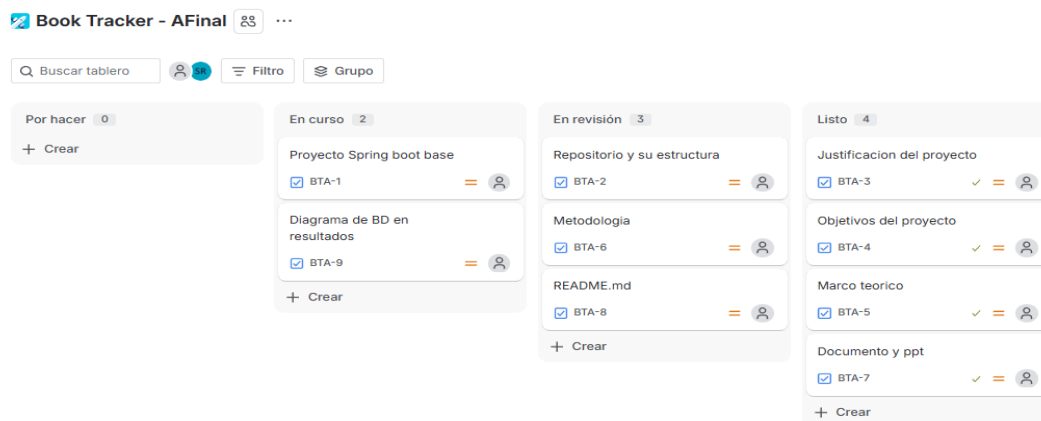
☐	Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución
☐	BTA-9 Diagrama de BD en resultados		Samir Rojas	Medium		Sin resolver
☐	☒ BTA-10 Implementacion de entidades		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-11 Creacion de repositorios		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-13 Seguridad		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-14 Documentacion y ppt		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-15 Recoleccion de evidencias		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-16 Uso de herramientas de gestion		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-17 Integración Jira-GitHub		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-18 Automatización Jira-GitHub		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-19 Incidentes y Bugs		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-20 Dockerización y Pipeline CI/CD		Samir Rojas	Medium	POR HACER	Sin resolver
☐	☒ BTA-21 Calidad de Código		Samir Rojas	Medium	POR HACER	Sin resolver

Nota. Elaboración propia

El Board o tablero representa visualmente el flujo de trabajo. Las tareas se muestran como tarjetas que se mueven entre diferentes estados. Permite conocer rápidamente tareas faltantes, cuáles están en desarrollo, cuáles ya terminaron.

Figura 38

Tablero en JIRA



Nota. Elaboración propia

El Backlog funciona como un repositorio de actividades pendientes o planificadas para el proyecto. En este caso, las tareas seleccionadas fueron incorporadas al Sprint 1 para ser desarrolladas durante el período establecido.

Figura 39

Backlog en JIRA

Book Tracker - AFinal ...

Q Buscar en el ba... [User Icon] Filtro [Icons]

☐ **BTA Sprint 1** 14 abr - 27 abr (9 actividades) 0 0 0 Completar sprint ...

☒ BTA-1 Proyecto Spring boot base	LISTO ✓	-	=	[User Icon]
☒ BTA-2 Repositorio y su estructura	LISTO ✓	-	=	[User Icon]
☒ BTA-3 Justificacion del proyecto	LISTO ✓	-	=	[User Icon]
☒ BTA-4 Objetivos del proyecto	LISTO ✓	-	=	[User Icon]
☒ BTA-5 Marco teorico	LISTO ✓	-	=	[User Icon]
☒ BTA-6 Metodologia	LISTO ✓	-	=	[User Icon]
☒ BTA-7 Documento y ppt	LISTO ✓	-	=	[User Icon]
☒ BTA-8 README.md	LISTO ✓	-	=	[User Icon]
☒ BTA-9 Diagrama de BD en resultados	LISTO ✓	-	=	[User Icon]

+ Crear

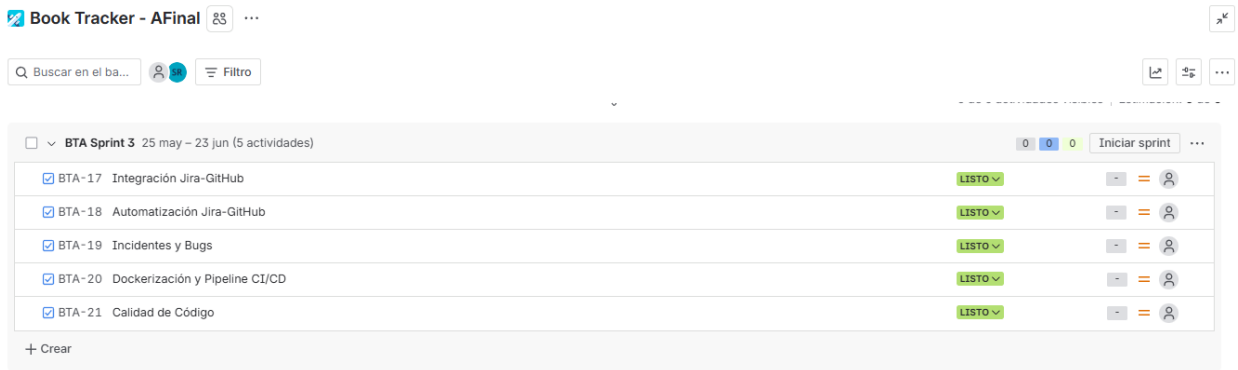
Book Tracker - AFinal ...

Q Buscar en el ba... [User Icon] Filtro [Icons]

☐ **BTA Sprint 2** 28 abr - 24 may (6 actividades) 0 0 0 Iniciar sprint ...

☒ BTA-10 Implementacion de entidades	LISTO ✓	-	=	[User Icon]
☒ BTA-11 Creacion de repositorios	LISTO ✓	-	=	[User Icon]
☒ BTA-13 Seguridad	LISTO ✓	-	=	[User Icon]
☒ BTA-14 Documentacion y ppt	LISTO ✓	-	=	[User Icon]
☒ BTA-15 Recoleccion de evidencias	LISTO ✓	-	=	[User Icon]
☒ BTA-16 Uso de herramientas de gestion	LISTO ✓	-	=	[User Icon]

+ Crear

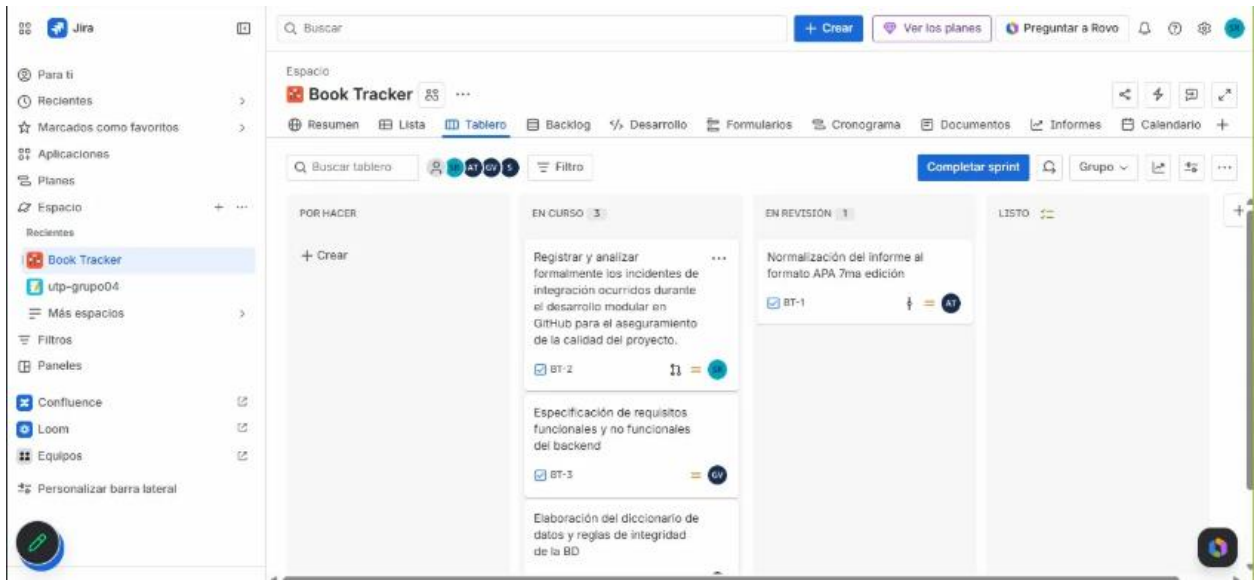


Nota. Elaboración propia

Jira sirve para organizar el trabajo, mientras que Git/GitHub sirve para guardar y controlar el código. Cuando los conectas, Jira puede saber automáticamente qué tarea está relacionada con qué código. Entonces, Jira se utilizó para la planificación y seguimiento general del proyecto, mientras que GitHub se empleó para el desarrollo del código. La integración entre ambas herramientas permite relacionar tareas, ramas, commits y Pull Requests, facilitando la trazabilidad del trabajo realizado por cada integrante del equipo.

Figura 40

Tarea en estado “en curso”

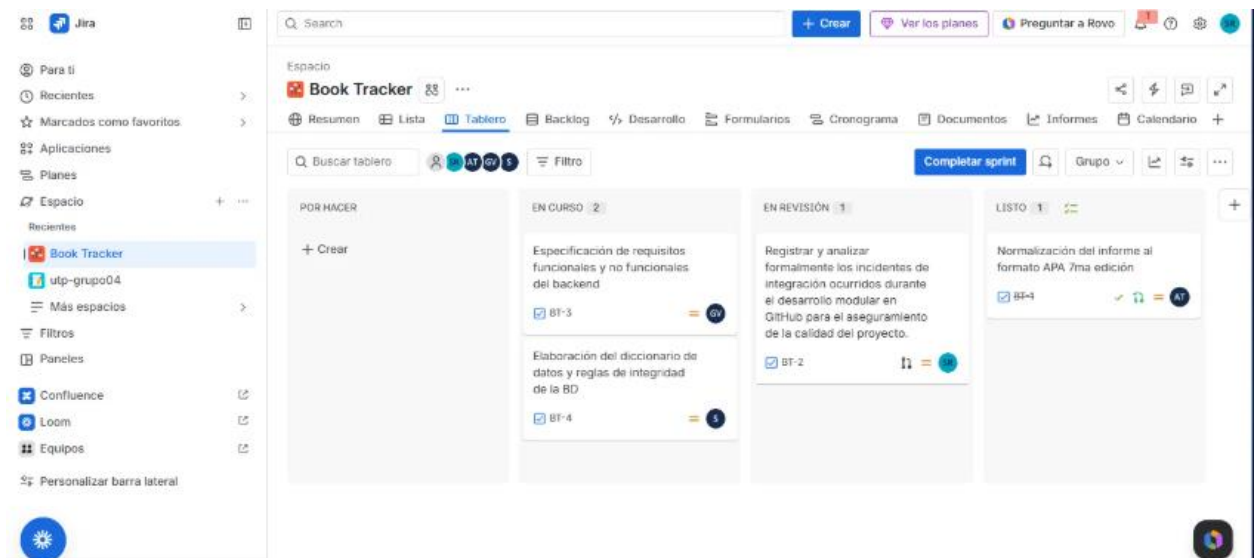


Nota. Elaboración propia

La tarea cambia automáticamente al estado “en curso” después de crear la rama correspondiente en GitHub.

Figura 41

Tarea en estado “en revisión”

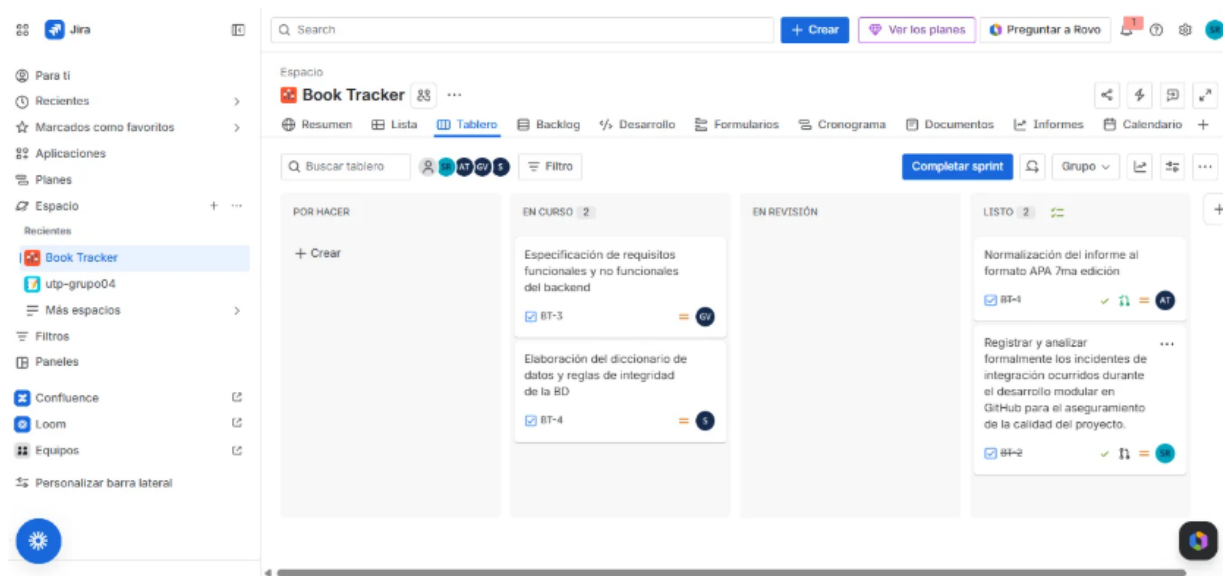


Nota. Elaboración propia

La tarea cambia automáticamente al estado “en revisión” cuando el desarrollador hace un Pull request en GitHub.

Figura 42

Tarea en estado “en listo”



Nota. Elaboración propia

La tarea cambia automáticamente al estado “en listo” cuando se hace el merge en GitHub.

7.4.1. Gestión de incidentes con Jira Service Management

La gestión de incidentes se realiza mediante la herramienta Jira Service Management, la cual permite administrar solicitudes, errores y eventos que afectan el correcto funcionamiento del sistema, facilitando su seguimiento y resolución dentro del equipo de desarrollo.

- **ITSM (Information Technology Service Management):** ITSM es un enfoque de gestión de servicios de TI que se centra en la planificación, entrega, operación y mejora continua de los servicios tecnológicos. Su objetivo principal es asegurar que los servicios de TI estén alineados con las necesidades del negocio y se mantenga la calidad operativa.
- **Incidente:** Un incidente es cualquier interrupción no planificada o degradación de un servicio que afecta el funcionamiento normal del sistema. En el contexto del proyecto Book Tracker, los incidentes representan errores técnicos que deben ser atendidos y solucionados por el equipo de desarrollo.
- **Creación del proyecto:** Para la gestión de incidencias se creó el proyecto en Jira Service Management denominado Book Tracker ITSM. Este proyecto permite centralizar el registro, seguimiento y resolución de los incidentes y bugs del sistema. Algunos de los tickets registrados durante el desarrollo fueron los siguientes:

INC-001: Error PostgreSQL

Incidente relacionado con fallos en la conexión o ejecución de consultas en la base de datos PostgreSQL.

BUG-001: No registra CREO_LISTA

Error en la funcionalidad de creación de listas de libros, donde no se persisten los datos correctamente.

BUG-002: No registra AGREGO_LIBRO

Problema en la funcionalidad de guardado de libros en listas que impide el registro en actividades cuando un libro es guardado en una lista.

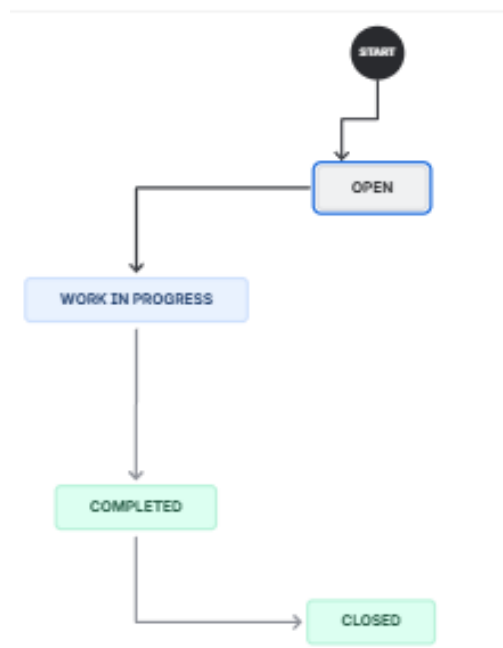
7.4.2. Automatización del flujo de incidencias

El flujo de gestión de incidencias en Jira Service Management fue automatizado para optimizar el seguimiento del ciclo de vida de cada ticket, reduciendo la intervención manual y asegurando la trazabilidad del proceso.

Los incidentes y bugs siguen el siguiente ciclo de vida:

Figura 43

Flujo de estados en Jira Service Management



Nota. Elaboración propia

Este flujo permite controlar el progreso de cada incidencia desde su creación hasta su cierre definitivo, garantizando que cada etapa sea validada antes de avanzar.

Se configuraron reglas en Jira Automatization para automatizar la transición de estados según eventos del repositorio:

Figura 44

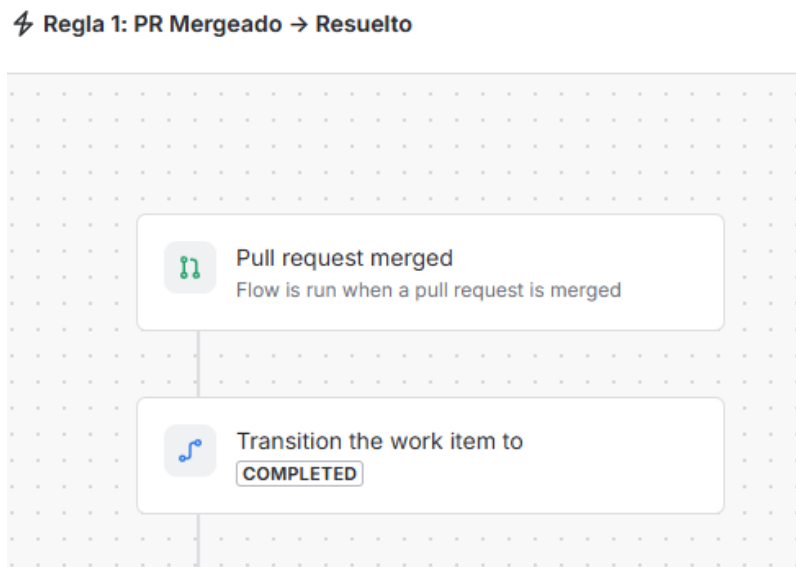
Reglas y triggers para automatización

Regla 1: PR Mergeado → Resuelto	AT Adriana Tapia	Global	7 days ago	☒	...
Regla 2: PR Creado → En desarrollo	AT Adriana Tapia	Global	7 days ago	☒	...

Nota. Elaboración propia

Figura 45

Regla cuando Pull Request is merged



Nota. Elaboración propia

7.4.3. Integración Jira-GitHub

La integración entre Jira y GitHub permite establecer una trazabilidad completa entre los tickets de incidencias y el desarrollo del código fuente, conectando directamente la gestión de tareas con el repositorio del proyecto.

El flujo implementado es el siguiente:

Ticket Jira->Branch->Commit->Pull Request->Merge

Este proceso asegura que cada cambio en el código esté asociado a un ticket específico, permitiendo un mejor control del desarrollo.

Por ejemplo, para el ticket con ID B123-6, se siguió el flujo mencionado. Primero, la creación de rama con el ID del incidente B123-6, luego, la realización de commits asociados al ticket, y de ahí la creación de Pull Request vinculado a Jira, PR asociado al ticket, y por último, merge del Pull Request en la rama correspondiente.

Jira detecta automáticamente la relación con GitHub mostrando el cambio de estado del incidente. Esto permite visualizar directamente en el ticket todo el historial de desarrollo asociado, mejorando la trazabilidad y el control del proyecto.

Figura 46

Listado de incidentes

Work	Assignee	Reporter	Priority	Status
B123-9 La librería de API OpenLib...	ALUMNO - SANDRO PIERO SALAZAR CA...	Adriana Tapia	Medium	OPEN
B123-8 Error 500 en /auth/authenti...	Samir Rojas	Gerardo Adrian Ra...	Medium	COMPLETED
B123-7 Al ejecutar el endpoint de ...	Gerardo Adrian Ramos Valdivia	Adriana Tapia	Medium	CLOSED
B123-6 No se registra actividad C...	Adriana Tapia	Gerardo Adrian Ra...	Medium	CLOSED

Nota. Elaboración propia

Figura 47

Incidente B123-6

No se registra actividad CREO_LISTA al crear una lista de lectura

Completed Automation

Gerardo Adrian Ramos Valdivia
View request in portal

Description
Al ejecutar el endpoint de creación de listas, la lista se almacena correctamente en la base de datos, pero no se genera el registro correspondiente en la tabla Actividades.

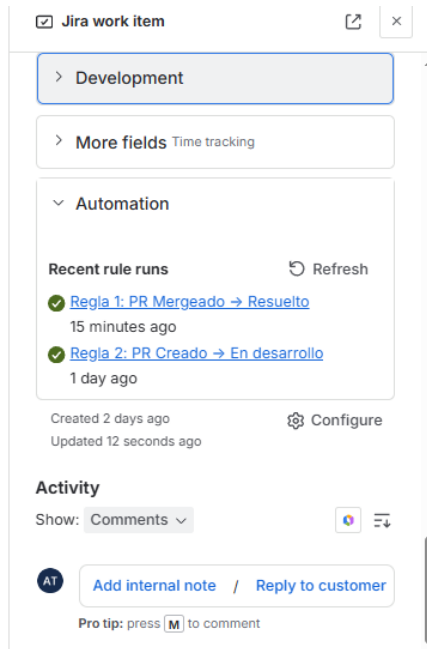
***Comportamiento esperado:** Registrar actividad CREO_LISTA.

***Comportamiento actual:** La actividad no aparece en el historial.

Nota. Elaboración propia

Figura 48

Reglas ejecutadas para B123-6



Nota. Elaboración propia

7.5. Resultados obtenidos

7.5.1. Funcionamiento del sistema

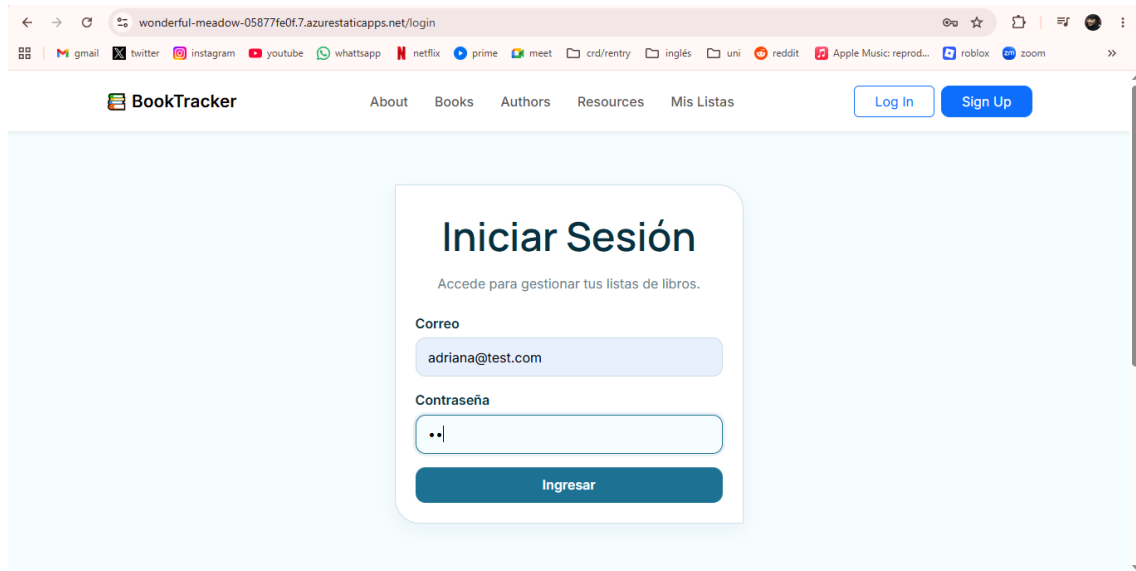
Como resultado del desarrollo e integración de todos los componentes de la plataforma, se obtuvo una aplicación web completamente funcional que permite facilitar a diferentes libros mediante una arquitectura cliente-servidor. El sistema integra un frontend desarrollado con Angular, una API REST implementada con Spring Boot y una base de datos relacional, permitiendo la comunicación entre todos los módulos de forma eficiente.

Durante las pruebas realizadas se verificó el correcto funcionamiento de los principales procesos implementados, entre ellos la autenticación de usuarios, la gestión de listas de libros, la administración de las listas, la creación de comentarios y la búsqueda de libros y autores. Asimismo, se comprobó que la aplicación responde correctamente a las solicitudes enviadas desde la interfaz gráfica hacia el backend y que la información es almacenada y recuperada correctamente desde la base de datos.

Los resultados obtenidos demuestran que la solución desarrollada cumple con los requerimientos funcionales establecidos durante la etapa de análisis y diseño del proyecto.

Figura 49

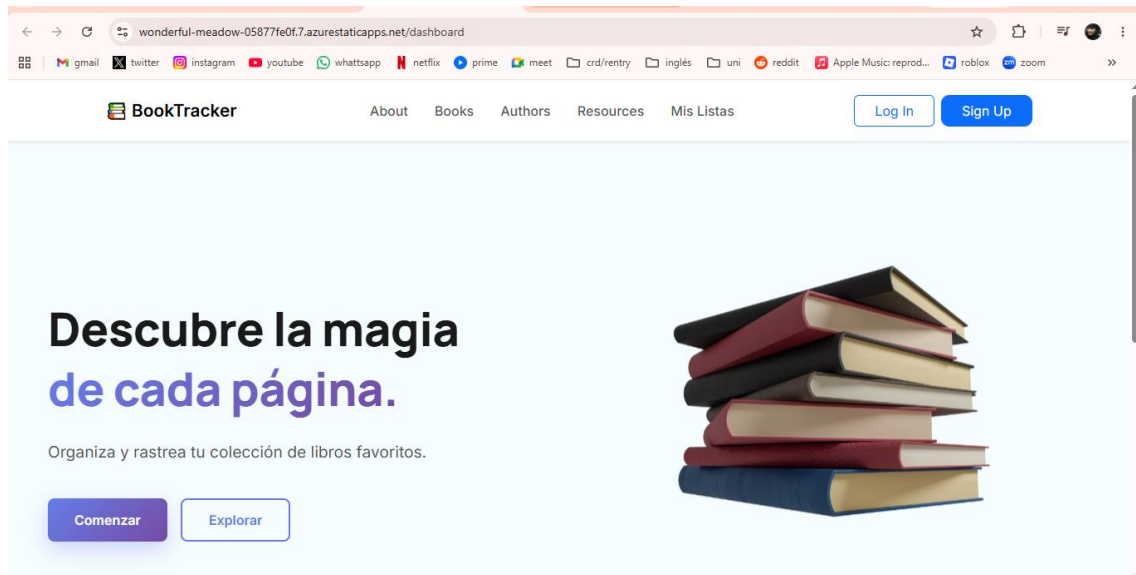
Pantalla de inicio de sesión



Nota. Elaboración propia

Figura 50

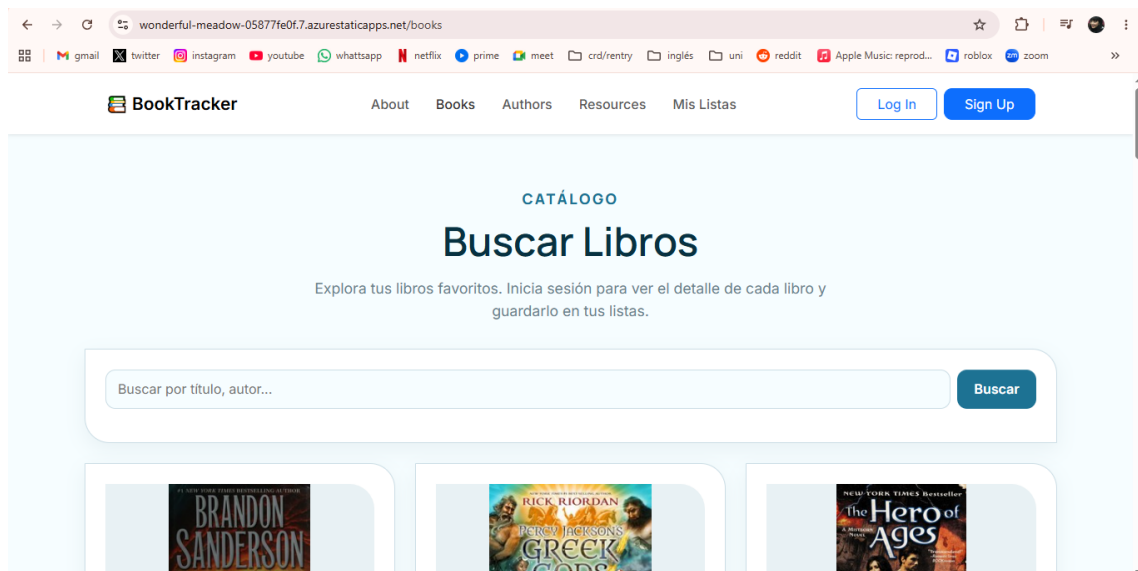
Pantalla de dashboard



Nota. Elaboración propia

Figura 51

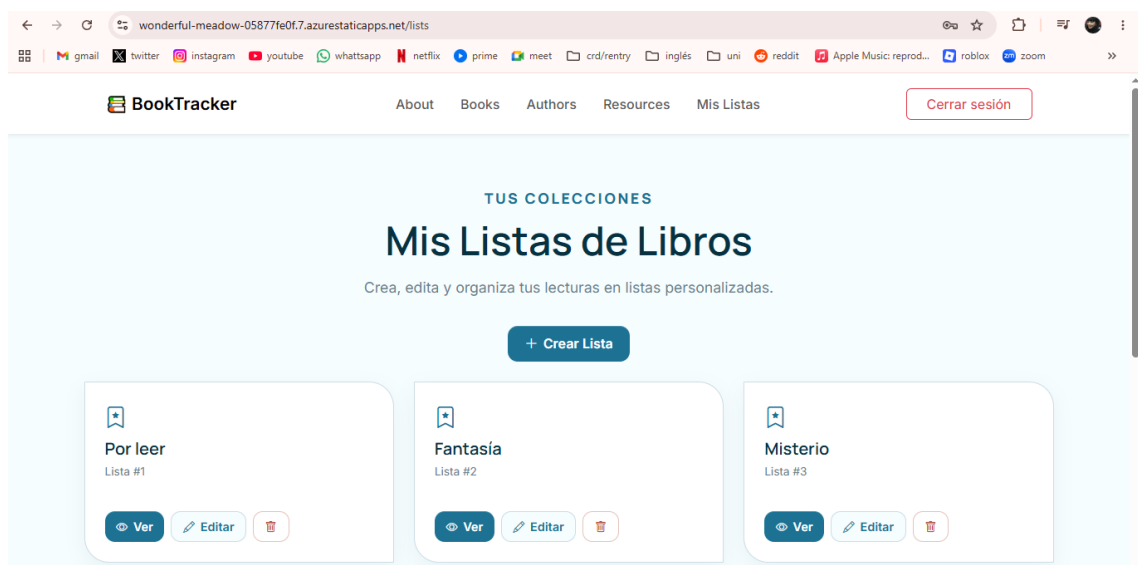
Pantalla de libros



Nota. Elaboración propia

Figura 52

Pantalla de listas de libros



Nota. Elaboración propia

7.5.2. Evaluación del pipeline DevOps

Durante la etapa media y final del proyecto se implementó un pipeline de Integración Continua y Despliegue Continuo (CI/CD), permitiendo automatizar las tareas de compilación, análisis de calidad, verificación de seguridad y despliegue de la aplicación.

El pipeline ejecuta automáticamente la construcción del proyecto, la ejecución de pruebas, el análisis de calidad mediante SonarCloud, el análisis de vulnerabilidades con Snyk, el escaneo de imágenes Docker mediante Trivy y el despliegue de la aplicación hacia los servicios de Azure.

La integración de estas herramientas permitió reducir tareas manuales, detectar posibles errores de manera temprana y garantizar que únicamente versiones verificadas fueran publicadas en el entorno de despliegue.

En conjunto, los resultados obtenidos evidencian que el flujo DevOps implementado cumple con los principios de automatización, integración continua y entrega continua establecidos para el desarrollo moderno de aplicaciones.

8. Conclusiones

El desarrollo del proyecto Book Tracker permitió implementar una aplicación web funcional basada en una arquitectura cliente-servidor, integrando un frontend desarrollado con Angular, un backend construido con Spring Boot y una base de datos alojada en Azure SQL Database. La solución desarrollada cumple con los objetivos planteados, proporcionando una plataforma para la gestión de cursos, contenidos y usuarios mediante servicios REST, garantizando una comunicación eficiente entre todos sus componentes.

La incorporación de herramientas de control de versiones y prácticas DevOps permitió optimizar el proceso de desarrollo del software. El uso de Git y GitHub facilitó el trabajo colaborativo y el seguimiento de los cambios realizados, mientras que GitHub Actions automatizó la compilación, ejecución de pruebas, análisis de calidad, construcción de imágenes Docker y publicación en Docker Hub. Asimismo, el despliegue del frontend en Azure Static Web Apps y del backend en Azure App Service permitió contar con un proceso de entrega continua confiable y automatizado.

La integración de herramientas orientadas a la calidad y seguridad, como SonarCloud, Snyk y Trivy, fortaleció el proceso de desarrollo al permitir identificar problemas de calidad, vulnerabilidades en las dependencias y posibles riesgos de seguridad en las imágenes Docker. Estas herramientas contribuyeron a mejorar la confiabilidad, mantenibilidad y seguridad de la aplicación, promoviendo la adopción de buenas prácticas de desarrollo.

Finalmente, la utilización de Jira Service Management y su integración con GitHub permitió gestionar de forma organizada las incidencias y actividades del proyecto, mejorando la planificación y el seguimiento del trabajo realizado. En conjunto, la implementación de prácticas de integración continua, despliegue continuo, contenerización, servicios en la nube y gestión de proyectos permitió obtener una solución tecnológica robusta, escalable y preparada para futuras ampliaciones y mejoras.

9. Referencias bibliográficas

Baddeley, A. (2015). Memoria humana: teoría y práctica. McGraw-Hill.

<https://dialnet.unirioja.es/servlet/libro?codigo=94880>

CERLALC-UNESCO. (2020). Lectura en papel vs. lectura en pantalla. Centro Regional para el Fomento del Libro en América Latina y el Caribe (Cerlalc). <https://www.cerlalc.org>

Cordón, J. A., & Jarvio, A. O. (2015). ¿Se está transformando la lectura en la era digital? Revista Interamericana de Bibliotecología, 38(2), 137–145.
<https://www.redalyc.org/pdf/1790/179038631005.pdf>

Cruces, F. (2017). ¿Cómo leemos en la sociedad digital? Fundación Telefónica.
https://www.lasnaves.com/wp-content/uploads/2017/08/Como_leemos_en_la_sociedad_digital_online.pdf

Delgado, P., Vargas, C., Ackerman, R., & Salmerón, L. (2018). Don't throw away your printed books: A meta-analysis on the effects of reading media on reading comprehension. Educational Research Review, 25, 23–38.
<https://www.sciencedirect.com/science/article/pii/S1747938X18300101>

Figuerola, M., Pitriqueo, P., & Zúñiga, S. (2016). Lectura digital en jóvenes. Educación y Tecnología, (9), 41–49. <https://revistas.umce.cl/index.php/edytec/article/view/528>

González-Ramírez, C. M., et al. (2020). Consumos lectores en adolescentes. Investigaciones sobre lectura, (13), 153–168.
https://www.researchgate.net/publication/341707209_Consumos_lectores_entre_adolescentes_Un_abordaje_comparado_desde_contextos_internacionales_El_caso_de_Polonia_Chile_y_Portugal

Ministerio de Cultura del Perú (2024). ¿Cómo vamos en el consumo de libros en el Perú? Indicadores de adquisición de libros impresos y digitales entre el 2016 y el 2022. Dirección del Libro y la Lectura. <https://www.perulee.pe>

Montoya, L., et al. (2019). Teorías pedagógicas en entornos digitales. Revista Información Científica. http://scielo.sld.cu/scielo.php?script=sci_abstract&pid=S1028-99332019000200241

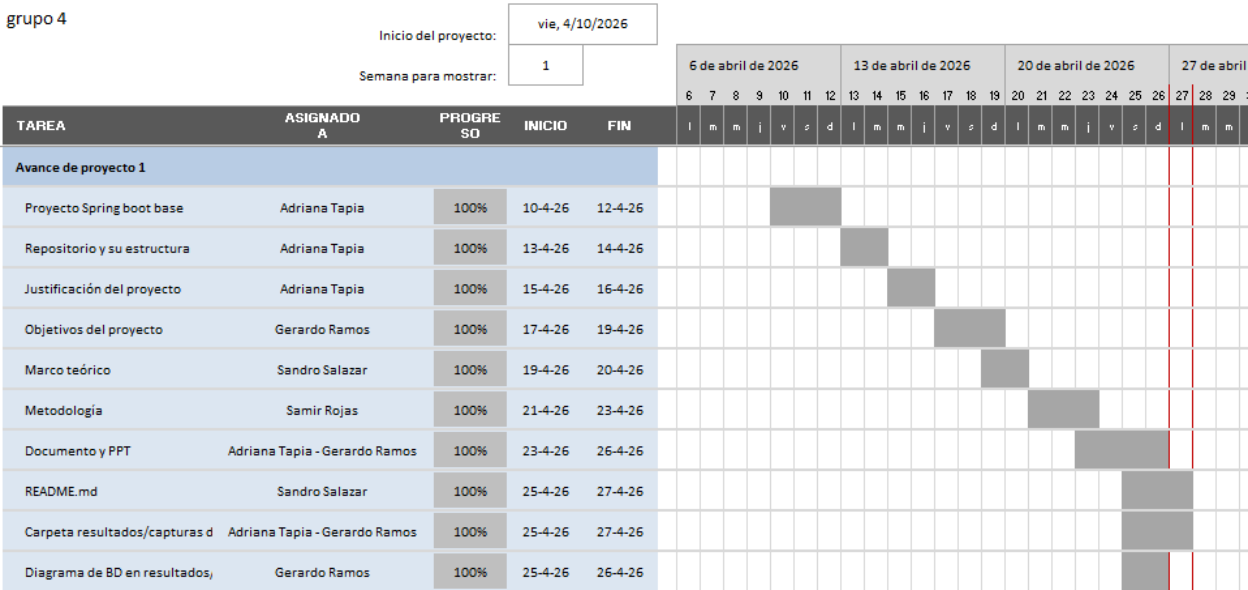
Murre, J. M., & Dros, J. (2015). Replication and Analysis of Ebbinghaus' Forgetting Curve. PLoS ONE, 10(7), e0120644. <https://doi.org/10.1371/journal.pone.0120644>

Organización para la Cooperación y el Desarrollo Económicos (OCDE). (2019). PISA 2018 Results (Volume I): What Students Know and Can Do. OECD Publishing. https://www.oecd.org/en/publications/pisa-2018-results-volume-i_5f07c754-en.html

Pérez, J., et al. (2023). Comprensión lectora en la era digital: revisión sistemática. Revista Andina de Educación. <https://www.redalyc.org/journal/7300/730077730001/html/>

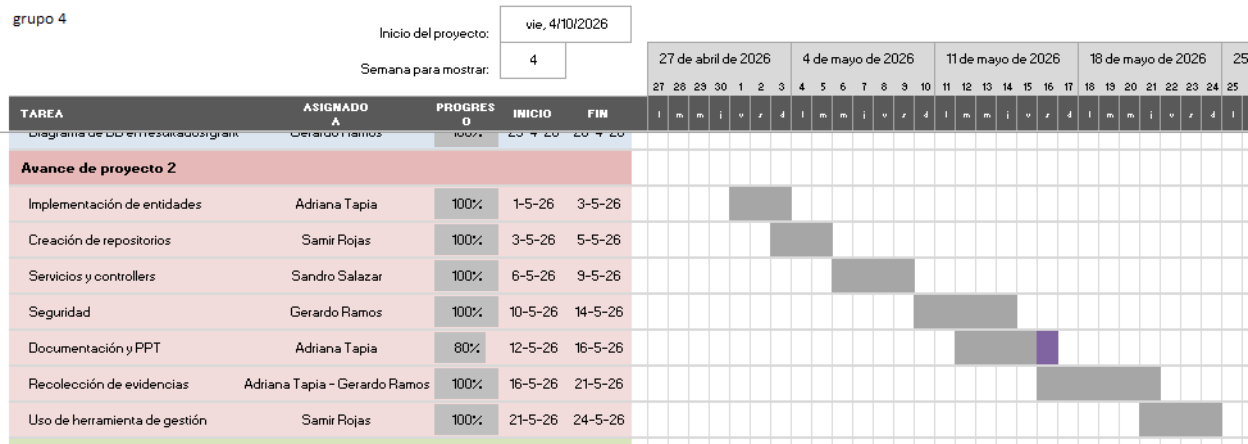
Anexos

Anexo 1



Anexo 2

Herramientas de desarrollo: gestor biblioteca



Anexo 3

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (develop)
$ git checkout -b feature/local-a
Switched to a new branch 'feature/local-a'

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a)
$ git status
On branch feature/local-a
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a)
$ git add .

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a)
$ git commit -m "test: cambio local A"
[feature/local-a 8a639fb] test: cambio local A
1 file changed, 1 insertion(+), 1 deletion(-)
```

Anexo 4

```

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (develop)
$ git checkout -b feature/local-b
Switched to a new branch 'feature/local-b'

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-b)
$ git status
On branch feature/local-b
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-b)
$ git add .

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-b)
$ git commit -m "test: cambio local B"
[feature/local-b 78638a3] test: cambio local B
1 file changed, 1 insertion(+), 1 deletion(-)

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-b)
$ git checkout feature/local-a
Switched to branch 'feature/local-a'

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a)
$ git merge feature/local-b
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.

```

Anexo 5

```

README.md ! X
README.md > # book-tracker ejemplo conflicto ver. B
You, hace 13 minutos | 1 author (You) | Aceptar cambio actual | Aceptar cambio entrante | Aceptar ambos cambios | Comparar cambios
1 <<<<<< HEAD (Cambio actual)
2 # book-tracker ejemplo conflicto ver. A
3 =====
4 # book-tracker ejemplo conflicto ver. B
5 >>>>>> feature/local-b (Cambio entrante)
6

```

Anexo 6

```
tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a|MERGING)
$ git status
On branch feature/local-a
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

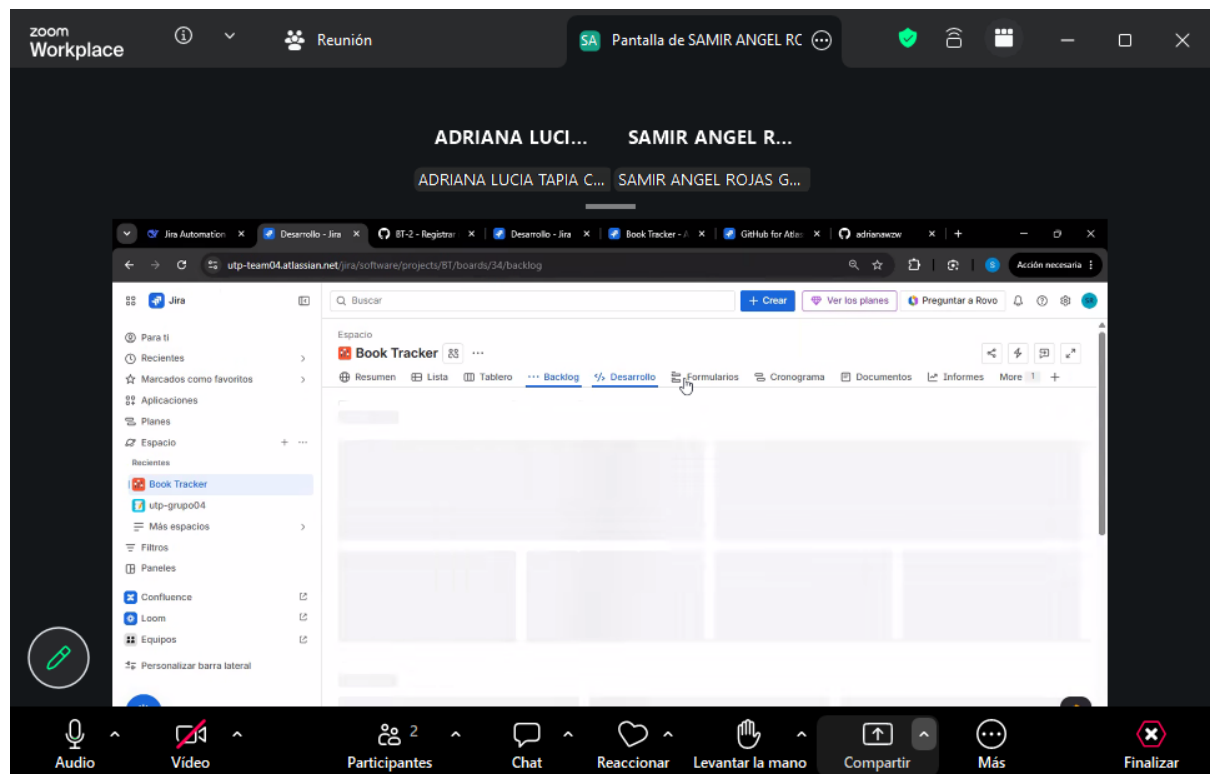
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a|MERGING)
$ git add .

tapia@SanIsidro MINGW64 /d/Proyectos/Proyecto-HerramientasDeDesarrollo/book-tracker (feature/local-a|MERGING)
$ git commit -m "fix: resolver conflicto local"
[feature/local-a 027526d] fix: resolver conflicto local
```

Anexo 7



Anexo 8

```

1  name: Integracion Continua BookTracker
2
3  on:
4    pull_request:
5      branches:
6        - develop
7        - main
8
9  jobs:
10   build-and-test:
11     runs-on: ubuntu-latest
12
13     defaults:
14       run:
15         working-directory: src/book-tracker
16
17     steps:
18       - name: 1. DescargarCodigoFuente
19         uses: actions/checkout@v4
20
21       - name: 2. ConfigurarEntornoJava17
22         uses: actions/setup-java@v4
23         with:
24           java-version: '17'
25           distribution: 'temurin'
26
27       - name: 3. CompilacionyPruebasUnitarias (JaCoCo Coverage)
28         run: mvn clean verify
29
30       - name: 4. AnalisisdeCalidaddeCodigo (SonarCloud)
31         env:
32           GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
33           SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
34         run: |
35           mvn sonar:sonar \
36             -Dsonar.projectKey=adrianawne/book-tracker \
37             -Dsonar.organization=adrianawne \
38             -Dsonar.token=${ secrets.SONAR_TOKEN }
39
40       - name: 5. Docker login
41         uses: docker/login-action@v3
42         with:
43           username: ${ secrets.DOCKER_USERNAME }
44           password: ${ secrets.DOCKER_PASSWORD }
45
46       - name: "6. Empaquetado: Construir Imagen Docker Multi-Stage"
47         run: docker build -t adrianawne/book-tracker:latest .
48
49       - name: 7. Escaneo de Vulnerabilidades de Imagen (Trivy)
50         uses: aquasecurity/trivy-action@master
51         with:
52           image-ref: 'adrianawne/book-tracker:latest'
53           format: 'table'
54           exit-code: '0' # <-- Cambiado de '1' a '0'
55           severity: 'HIGH,CRITICAL'
56
57       - name: "8. Despliegue: Subir Imagen a Docker Hub"
58         run: docker push adrianawne/book-tracker:latest

```

Anexo 9

Anexo 10

Para ti

Recientes >

Marcados como fav... >

Aplicaciones

Espacio + ...

Recientes

Book Tracker Ser... ..

Empezar

Resumen

Colas

Bandeja d... **NOVEDAD**

Actividad en varios e...

Solicitudes de servicio

Incidentes + ...

Default

Open incidents 3

My incidents 1

Mayor incidents 0

Resolved inci... 0

Ver todas las c... >

Plantilla **NOVEDAD**

Informes

Operaciones

Base de conocimient...

Clientes

Espacio / Book Tracker Service Desk / Incidentes

Open incidents

☆ ↗

Lista +

Buscar actividad
 Tipo de solicitud ▼ Estado ▼ Persona asignada ▼ Más filtros ▼

Priorización ↗

5 actividades

☐	Clave	Resumen	Informador	Persona asignada	Estado	Creada	Time to resolution ↑
☐	#123-5	Incidente ramas	Adriana Tapia	Sin asignar	CERRADA ▼	16/jun/26	Ayer 10:28 ⌚
☐	#123-6	No se registra actividad CREO_LISTA al crear una lista de lectura.	Gerardo Adrian Ramos Valdivia	AT Adriana Tapia	CERRADA ▼	17/jun/26	Hoy 15:40 ⌚
☐	#123-7	Al ejecutar el endpoint de agregar libro a lista, se ejecuta correctamente. Sin embargo, no se genera ni registra en la tabla Actividad, la acción correspondiente.	Adriana Tapia	GV Gerardo Adrian Ramos...	CERRADA ▼	18/jun/26	Mañana 13:00 ⌚
☐	#123-8	Error 500 en /auth/authenticar con JSON mal formado	Gerardo Adrian Ramos Valdivia	SR Samir Rojas	CERRADA ▼	22/jun/26	29 jun 13:00 ⌚
☐	#123-9	La librería de API OpenLibrary no responde	Adriana Tapia	AC ALUMNO - SANDRO P...	CERRADA ▼	23/jun/26	30 jun 10:44 ⌚

Anexo 11

```
sandro@MacBook-Air-de-SANDRO book-tracker % clear

sandro@MacBook-Air-de-SANDRO book-tracker % git branch

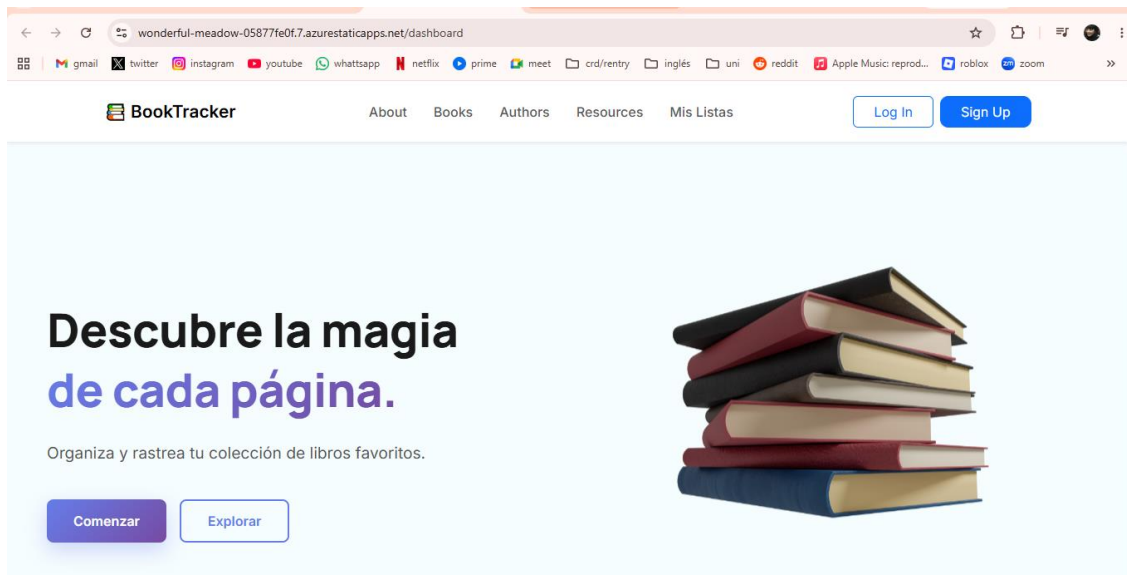
* B123-9
develop
main
sandro@MacBook-Air-de-SANDRO book-tracker % git log --oneline -3

4b28921 (HEAD -> B123-9, origin/B123-9) B123-9 fix: BUG-003 La libreria de API OpenLibrary no responde
ee141e1 (origin/develop, develop) Merge pull request #52 from adrianawzw/B123-8
98ccf4d (origin/B123-8) B123-8 fix: BUG-003 Agregar manejo de JSON mal formado en GlobalExceptionHandler
sandro@MacBook-Air-de-SANDRO book-tracker % git status

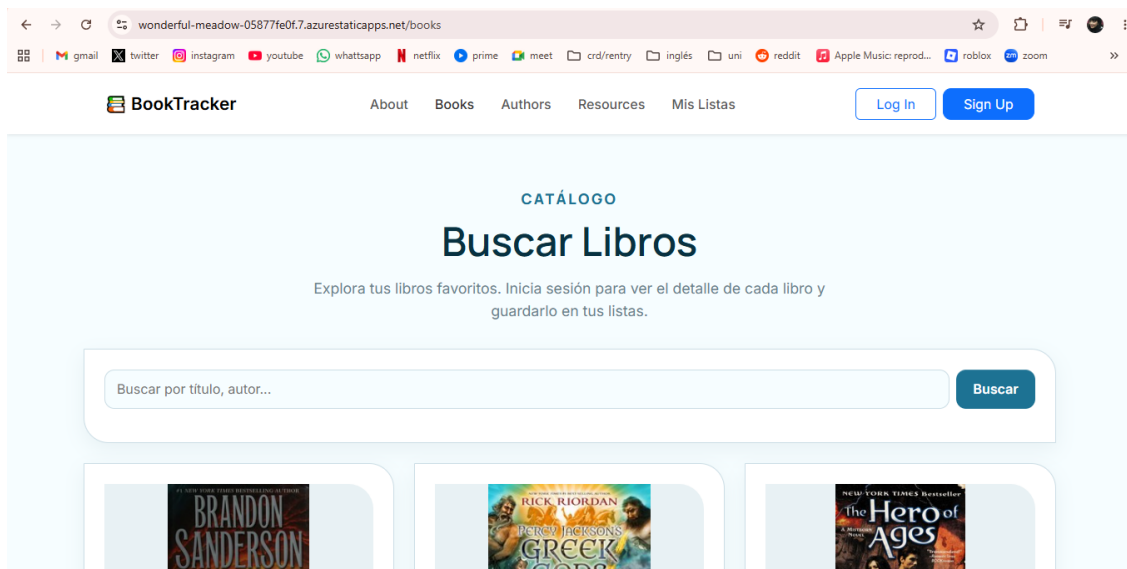
On branch B123-9
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .DS_Store
        src/.DS_Store

nothing added to commit but untracked files present (use "git add" to track)
```

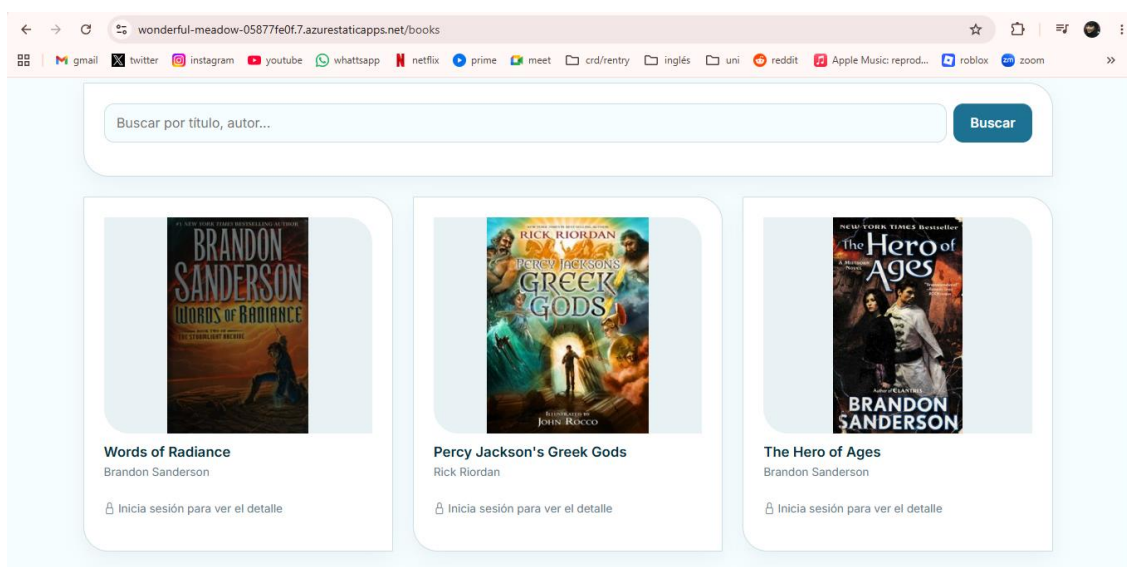
Anexo 12



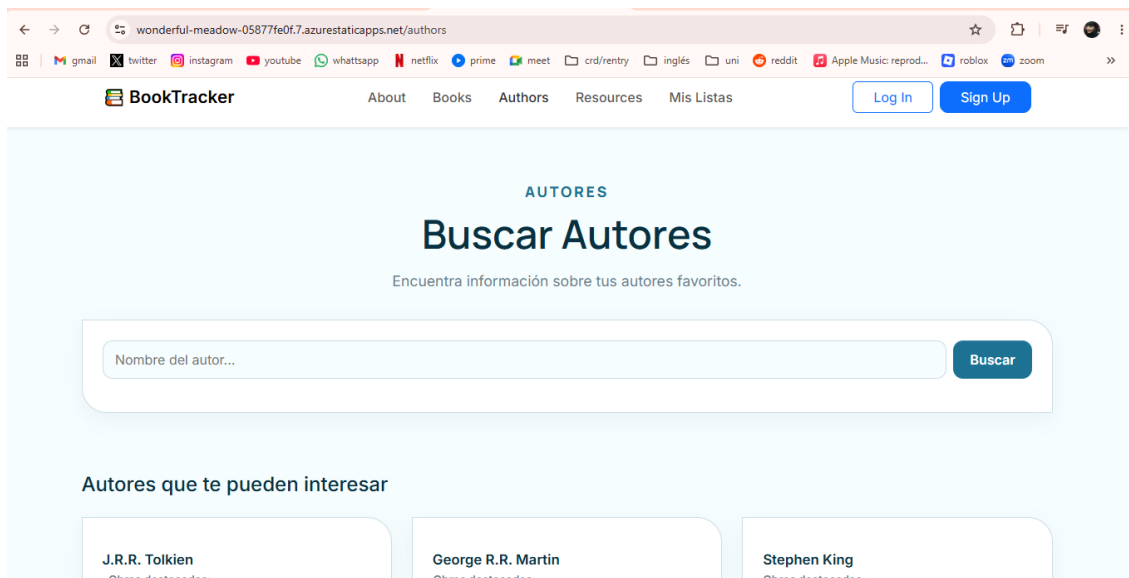
Anexo 13

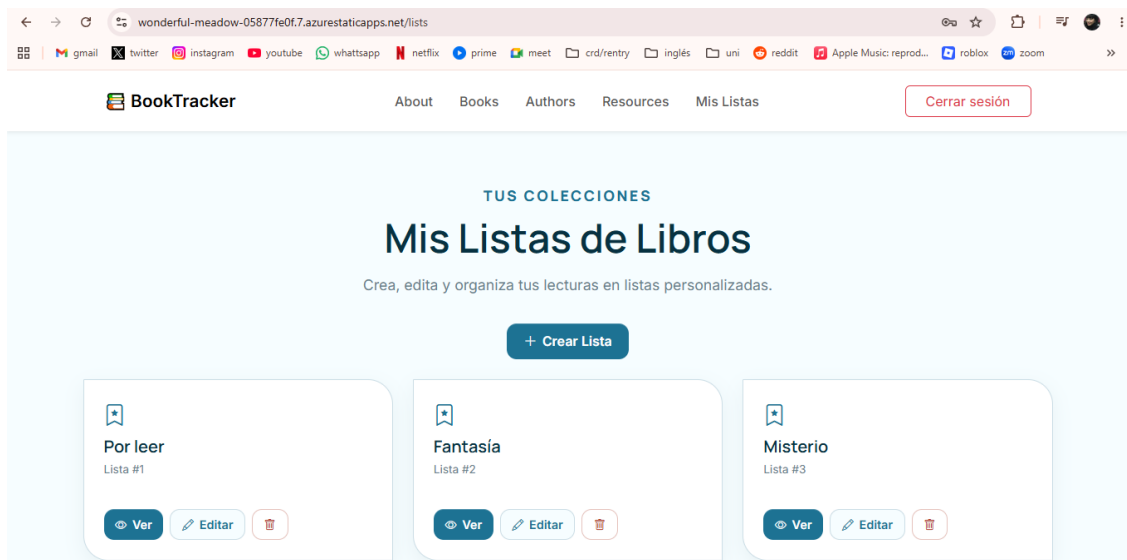
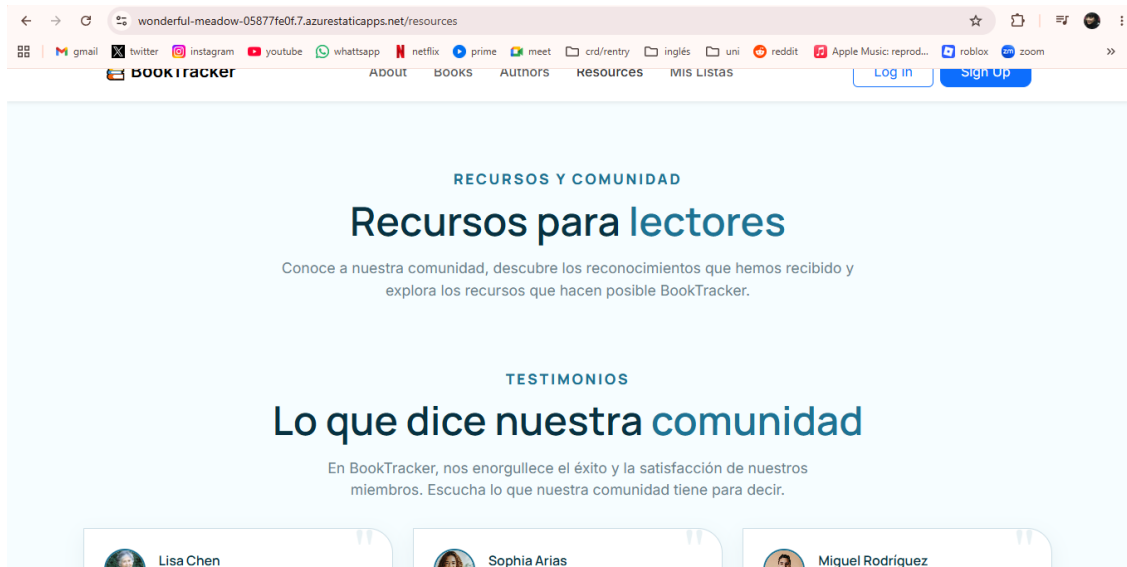


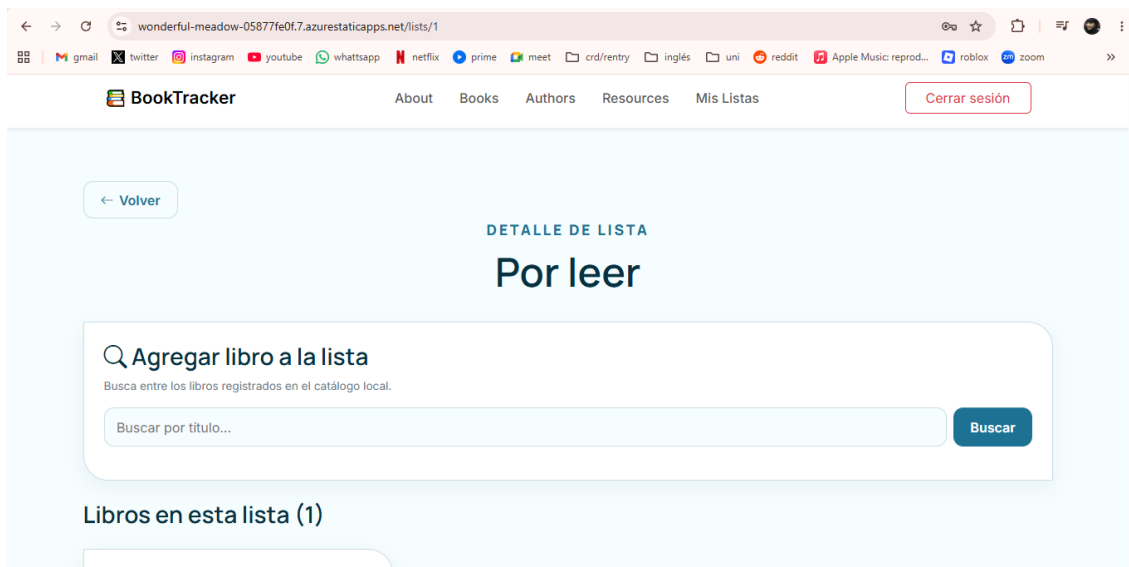
Anexo 14



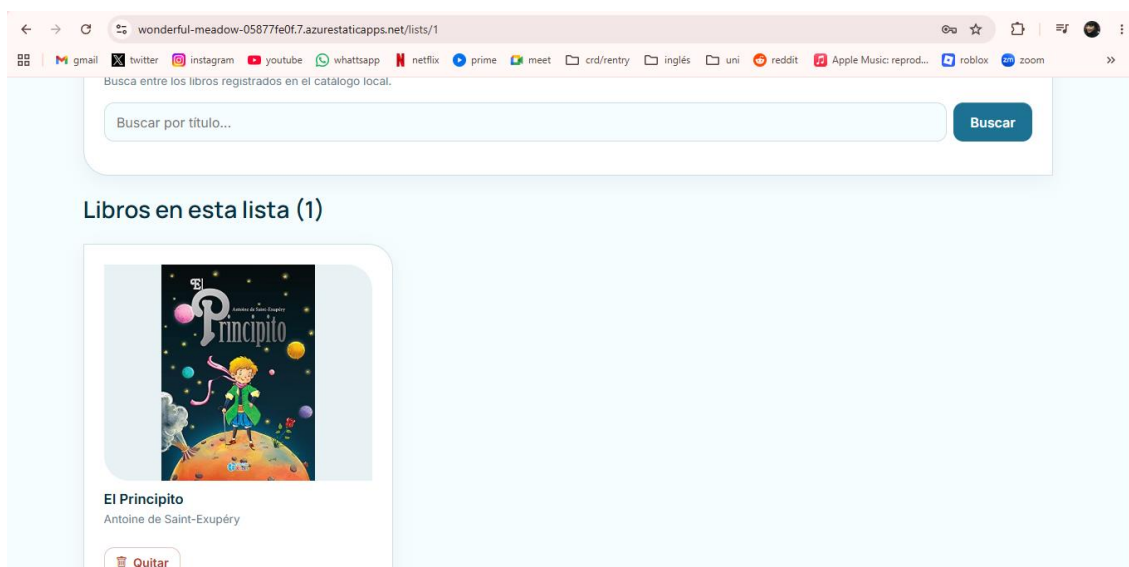
Anexo 15



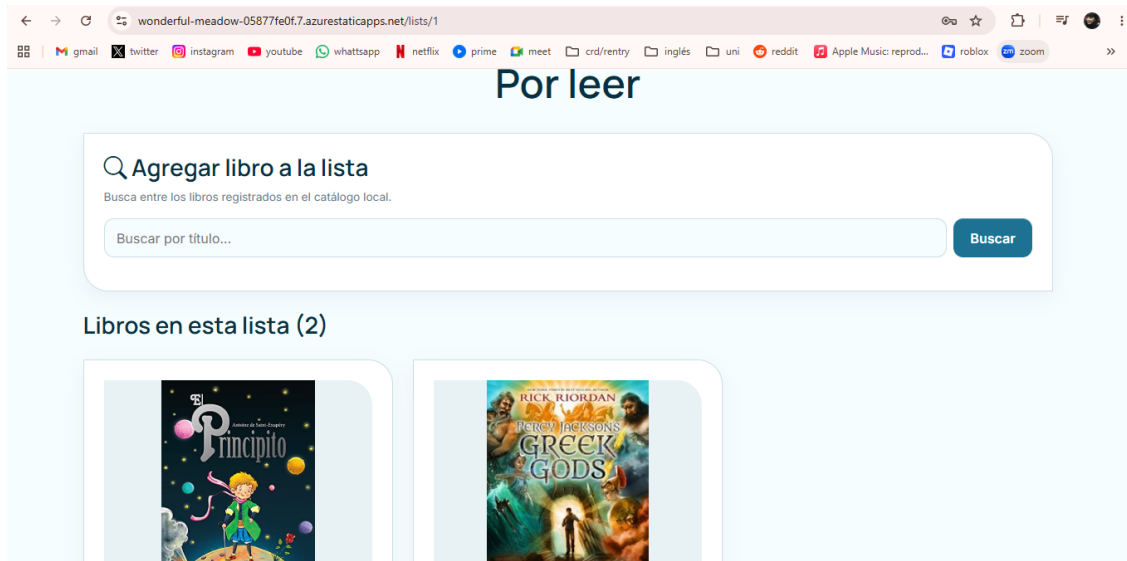




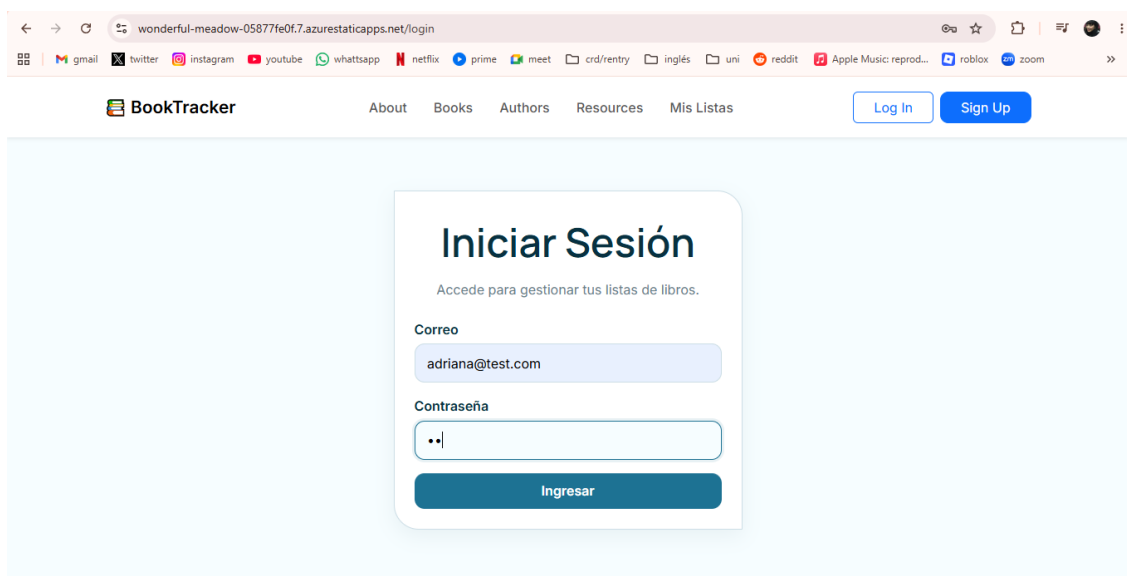
Anexo 17



Anexo 18



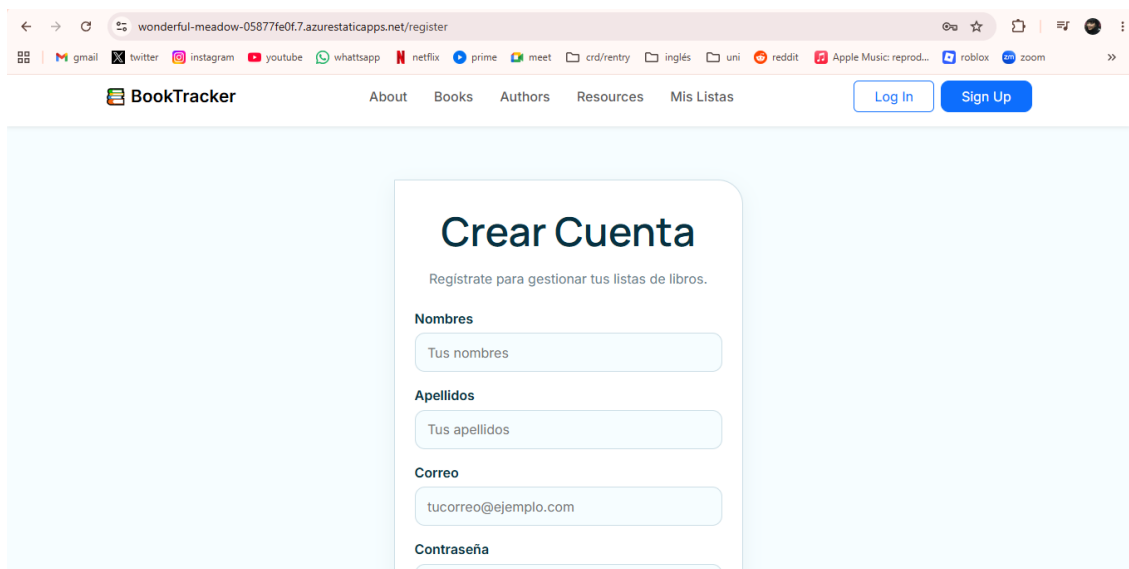
Anexo 19



Anexo 20



Anexo 21



Anexo 22

impl

▼ Implementación

Espacios de implementación

Centro de implementación

```
2026-07-15T15:09:40.1776761Z Opening named pipe /podr/container/pipe/702d679b9927_book-tracker-
api_main/stderr_78033b6705d546d683478c5459d5db11 for reading in non-blocking mode.
2026-07-15T15:09:40.1781288Z Successfully opened named pipe: /podr/container/pipe/702d679b9927_book-tracker-
api_main/stderr_78033b6705d546d683478c5459d5db11.
2026-07-15T15:09:40.1786003Z Successfully removed non-blocking flag from /podr/container/pipe/702d679b9927_book-tracker-
api_main/stderr_78033b6705d546d683478c5459d5db11.
2026-07-15T15:09:40.1807179Z Creating container with image: index.docker.io/adrianawzw/book-tracker:latest from registry:
index.docker.io and fully qualified image name: index.docker.io/adrianawzw/book-tracker:latest
2026-07-15T15:09:41.1978220Z Starting container: 702d679b9927_book-tracker-api_main.
2026-07-15T15:09:41.4818461Z Starting watchers and probes.
2026-07-15T15:09:41.5396618Z Starting metrics collection.
2026-07-15T15:09:41.5512420Z Container is running.
2026-07-15T15:09:41.5513664Z State: Starting, Action: CreatingContainer, LastError: , LastErrorTimestamp: 01/01/0001 00:00:00,
LastErrorDetails: , Details: Container book-tracker-api_main successfully created and is running., DetailsLevel: INFO
2026-07-15T15:09:41.6552242Z Container start method finished after 9446 ms.
2026-07-15T15:09:41.6555663Z State: Starting, Action: InitiatingSiteWarmupProbe, LastError: , LastErrorTimestamp: 01/01/0001
00:00:00, LastErrorDetails: , Details: Container book-tracker-api_main successfully created and is running., DetailsLevel: INFO
2026-07-15T15:09:41.7072171Z State: Starting, Action: WaitingForSiteWarmupProbeSuccess, LastError: , LastErrorTimestamp:
01/01/0001 00:00:00, LastErrorDetails: , Details: Pinging warmup path to ensure container is ready to receive requests.,
DetailsLevel: INFO
2026-07-15T15:09:54.9168215Z Site startup probe succeeded after 13.1938998 seconds.
2026-07-15T15:09:55.0372286Z State: Starting, Action: WarmupProbeSucceeded, LastError: , LastErrorTimestamp: 01/01/0001
00:00:00, LastErrorDetails: , Details: Site startup probe succeeded after 13.1938998 seconds., DetailsLevel: INFO
```